INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, SRI CITY

Course: Machine Learning (Assignment Evaluation)

Empirical Evaluation of Machine Learning Paradigms

Supervised Classification (CTG), Regularized Regression (Parkinson's), and Unsupervised Clustering & Representation Learning (Netflix)

Author Details & Submission Metadata	
Student Name(s)	Vinod Katevarapu, Moulidharan R
Roll Number(s)	S20240010256, S20240010152
Submission Date	4 November 2026

Abstract

This report provides an empirical investigation into three foundational paradigms of machine learning. In Question 1 (Classification), we evaluate K-Nearest Neighbours, Decision Trees, Support Vector Machines (RBF), and a deep 1D Convolutional Neural Network on the UCI Cardiotocography dataset (2,126 fetal heart recordings) using stratified 80/20 splitting. Mitigating extreme class imbalance via macro-averaged metrics, the RBF-SVM achieves the leading macro ROC-AUC of 0.9696, with bootstrap significance analysis establishing its parity with KNN and the CNN while outperforming Decision Trees. In Question 2 (Regression), we predict Parkinson's disease severity (total UPDRS) across 42 telemonitored patients (5,875 recordings). We demonstrate mathematically and empirically the convergence of scratch batch Gradient Descent to closed-form OLS (MSE 93.94 vs 93.90), show extreme multicollinearity (VIF 1.0e6), expose data leakage in random row splits (R^2 collapses from +0.9774 to −1.1656 for a Random Forest), and evaluate regularized polynomial pipelines. In Question 3 (Clustering & PCA), from-scratch Lloyd's K-Means and spectral PCA are implemented and validated against scikit-learn (ARI ≥ 0.9985). On 8,807 Netflix titles, a soft elbow at K=4 within the required grid (K=3 on the extended grid) separates movies from TV series, though silhouette scores are low (0.149), indicating overlapping clusters. PCA reveals that PC1 (format) captures 28.0% and PC2 (recency) captures 14.4% of total variance (42.4% cumulatively in 2 components).

Key Results at a Glance

Question	Headline result
Q1 — Best classifier (ROC)	SVM-RBF: macro ROC-AUC 0.9696, Pathologic AUC 0.9889, macro-F1 0.8450, accuracy 0.9178
Q1 — Significance	Only the Decision Tree is significantly worse (ΔAUC −0.0536, 95% CI [−0.0906, −0.0220]); KNN and CNN are statistically on par with the SVM
Q1 — Clinical sensitivity	1D-CNN has the highest Pathologic recall (94.29%, 33/35) at the cost of Normal recall (89.76%)
Q2 — Optimisation	Scratch gradient descent reaches training MSE 93.94 vs. 93.90 for closed-form OLS
Q2 — Leakage	Random Forest R^2 = +0.9774 with random-row split vs. −1.1656 with patient-disjoint split
Q2 — Regularisation	Lasso keeps 2 of 19 features; no model reliably beats the mean predictor on unseen patients (patient-wise CV)
Q3 — K-Means	Scratch implementation matches scikit-learn (ARI ≥ 0.9985); soft elbow at K = 4 (K = 3 on extended grid)
Q3 — PCA	PC1 28.0%, PC2 14.4% (42.4% in 2 PCs); 3 PCs retain 54.1% and reproduce the full-space clustering (ARI 0.9934)

Requirement Traceability Matrix

The table below maps every item of the assignment sheet to the section, table and figure of this report that addresses it, so that each requirement can be verified directly.

Assignment item	Where addressed in this report
Q1 (a) KNN with cross-validation	§2.2; Table 2.2; Figure 2.6 (left)
Q1 (b) Decision Tree	§2.2; Table 2.2; Figure 2.6 (right)
Q1 (c) SVM with RBF kernel	§2.2; Table 2.2; Figure 2.7
Q1 (d) Deep CNN (bonus)	§2.2; §2.3; Table 2.6; Figure 2.8
Q1 (i) 80% / 20% train–test split	§2.1 (stratified; 1,700 train / 426 test)
Q1 (ii) Confusion matrices	§2.3; Figures 2.3 and 2.4
Q1 (iii) Accuracy, precision, recall, F1, ROC-AUC	Tables 2.3, 2.4, 2.5; Figure 2.5
Q1 (iv) Overfitting and model complexity	§2.6; Table 2.7; Figures 2.6–2.9
Q1 (v) Best classifier by ROC analysis	§2.4 and §2.5 (SVM-RBF; bootstrap significance)
Q2 (a) Linear regression with gradient descent	§3.3; Table 3.3; Figures 3.4, 3.5
Q2 (b) Polynomial regression	§3.4; Table 3.5; Figures 3.6, 3.7
Q2 (c) Ridge and Lasso regression	§3.5; Figures 3.8–3.11
Q2 (i) MSE, RMSE and R^2 for each model	§3.6; Table 3.7; Table 3.8; Figure 3.12
Q2 (ii) Overfitting and underfitting cases	§3.4; Table 3.6; Figures 3.6, 3.7; §3.1 (leakage)
Q2 (iii) Cross-validation for regularization parameter	§3.5; Figures 3.8, 3.11
Q3 (a) K-Means from scratch; K = 2…6; WCSS; elbow	§4.2, §4.3; Tables 4.3, 4.5; Figures 4.1, 4.2
Q3 (b) PCA via covariance and eigendecomposition	§4.4; Tables 4.7, 4.8; Figure 4.4
Q3 (b) 2 and 3 principal components; visualization	§4.4, §4.5; Tables 4.8, 4.10; Figures 4.5–4.7
Submission: report (PDF) and code (zip)	This report; code archive structure in Appendix E

Report Organisation

Section	Content
1. Introduction	Datasets, rationale, experimental protocol, evaluation metrics, notation
2. Q1 — Classification	CTG data, four classifiers, confusion matrices, ROC and bootstrap analysis, overfitting study
3. Q2 — Regression	Patient-wise evaluation and leakage, multicollinearity, gradient descent, polynomial, Ridge and Lasso, benchmark
4. Q3 — Clustering and PCA	Netflix feature engineering, K-Means from scratch, elbow, PCA from scratch, projections and loadings
5. Discussion	Complexity control, evaluation protocol, scaling, practical recommendations
6. Conclusions and limitations	Synthesis of findings and explicit limitations
7. References; Appendices A–E	Environment, algorithm listings, derivations, glossary, submitted files

Contents

1. Introduction and Problem Formulation . . . 4
1.1 Benchmark Datasets . . . 4
1.2 Rationale for Dataset Selection . . . 4
1.3 Experimental Protocol and Reproducibility . . . 4
1.4 Evaluation Metrics . . . 4
1.5 Notation . . . 5
2. Q1 — Multiclass Classification on Cardiotocography . . . 6
2.1 Dataset Description, Preprocessing, and Anti-Leakage Safeguards . . . 6
2.2 Classifiers, Hyper-parameter Optimization and Model Architecture . . . 7
2.3 Benchmark Performance and Confusion Matrix Analysis . . . 8
2.4 Receiver Operating Characteristic (ROC) and Bootstrap Significance Analysis . . . 9
2.5 ROC-Based Conclusion . . . 11
2.6 Overfitting and Model Complexity Analysis . . . 11
2.7 Summary of Q1 Findings . . . 13
3. Q2 — Continuous Regression on Parkinson's Telemonitoring . . . 14
3.1 Dataset Architecture, Subject Disjointness, and Data Leakage . . . 14
3.2 Multicollinearity and Near-Exact Linear Dependencies . . . 15
3.3 Linear Regression with Gradient Descent from Scratch . . . 16
3.4 Polynomial Regression, Bias–Variance Tradeoff, and Overfitting . . . 17
3.5 Regularisation: Ridge, Lasso, and Coefficient Dynamics . . . 19
3.6 Benchmark Synthesis and the Honest Reality of UPDRS Prediction . . . 20
3.7 Summary of Q2 Findings . . . 22
4. Q3 — Unsupervised K-Means Clustering and PCA from Scratch . . . 23
4.1 Dataset Cleaning, Feature Engineering, and Continuous Scaling . . . 23
4.2 Vectorised K-Means from Scratch and scikit-learn Validation . . . 23
4.3 WCSS Analysis, Elbow Curve, and Cluster Profiling . . . 24
4.4 Principal Component Analysis (PCA) from Scratch . . . 26
4.5 2-D / 3-D Projections, Loadings Interpretation, and K-Means in PCA Space . . . 27
4.6 Summary of Q3 Findings . . . 29
5. Cross-Cutting Discussion . . . 30
5.1 Overfitting, Underfitting and Model Complexity . . . 30
5.2 Evaluation Protocol Matters More than Model Choice . . . 30
5.3 The Role of Feature Scaling . . . 30
5.4 Practical Recommendations . . . 30
6. Overall Conclusions and Scientific Limitations . . . 31
7. References . . . 31
Appendix A: Computational Environment and Runtimes . . . 33
Appendix B: Algorithm Listings . . . 33
Appendix C: Supporting Derivations . . . 33
Appendix D: Glossary of Terms and Abbreviations . . . 34
Appendix E: Submitted Code and Output Files . . . 34

1. Introduction and Problem Formulation

Modern machine learning spans diverse problem representations, data modalities, and optimization surfaces. This project investigates three distinct challenges that together cover the core of an introductory machine learning course: supervised classification, supervised continuous regression, and unsupervised geometric clustering with representation learning. Each task was constructed to probe fundamental principles: model capacity and the bias–variance tradeoff, structural data leakage, optimization convergence, the effect of feature scaling, the curse of dimensionality, and algorithmic fidelity of from-scratch implementations.

1.1 Benchmark Datasets

Table 1.1: Summary of Assignment Benchmark Datasets

Task & Dataset	Source	Instances / Features	Target / Objective	Methodological Focus
Q1: Multiclass Classification	UCI ID 193 (Cardiotocography)	2,126 recordings; 21 continuous features	NSP (fetal state: Normal, Suspect, Pathologic)	Class imbalance, macro metrics, 1D-CNN vs tabular models, bootstrap ROC CIs
Q2: Continuous Regression	UCI ID 189 (Parkinson's Telemonitoring)	5,875 observations; 19 voice/biomedical features; 42 patients	`total_UPDRS` (clinician motor/total rating, scale 0–176)	Patient-wise splitting, batch GD from scratch, VIF multicollinearity, polynomial regularisation
Q3: Clustering & Dimensionality Reduction	Kaggle / Shivamb (Netflix Catalog)	8,807 media titles; 66 engineered dimensions	Unsupervised taxonomy discovery and orthogonal latent projection	K-Means from scratch, covariance PCA from scratch, elbow/silhouette, validation vs scikit-learn

1.2 Rationale for Dataset Selection

- **Cardiotocography (CTG)** was selected because fetal heart monitoring presents severe clinical asymmetry: failing to detect a Pathologic fetus (a false negative) carries grave obstetric consequences. The acute class imbalance (77.8% Normal vs. 8.3% Pathologic) exposes the failure modes of naive overall accuracy and justifies macro-averaged metrics and ROC analysis.
- **Parkinson's Telemonitoring** represents longitudinal medical signals in which about 140 measurements per patient originate from only 42 individuals. Naive random-split cross-validation leaks patient identity, producing false optimism. Enforcing patient-disjoint partitions gives an honest estimate of clinical generalizability.
- **Netflix Catalog** embodies real-world multi-attribute metadata. Turning discrete and text attributes into continuous vector representations requires careful feature engineering, giving a realistic test-bed for from-scratch K-Means and PCA.

1.3 Experimental Protocol and Reproducibility

All experiments follow a common protocol designed to avoid optimistic estimates:

- **Fixed seeds.** Every stochastic step uses `random_state = 42` (the CNN additionally reports seeds 42–46) so that splits, initialisations and bootstrap draws are reproducible.
- **Held-out evaluation.** Hyper-parameters are tuned only by cross-validation on the training partition; the test partition is touched once per final model. In Q2 the partition is subject-wise rather than row-wise.
- **Leakage control.** Scalers are fitted inside `sklearn.pipeline.Pipeline` objects so that they only ever see training folds; sibling targets and identifiers (`CLASS`, `motor_UPDRS`, `subject#`) are removed from the inputs.
- **Validation of from-scratch code.** Gradient descent is compared with the closed-form solution; K-Means and PCA are compared numerically with their scikit-learn counterparts.
- **Honest reporting.** Where results are weak or non-significant (Q2 negative R^2, Q3 low silhouette, Q1 non-significant AUC differences) this is stated explicitly rather than hidden.

1.4 Evaluation Metrics

For classification with classes $k = 1 \ldots K$, the confusion matrix entries are denoted TP_k, FP_k, FN_k. Per-class precision and recall, and the F1-score, are

$$Precision_k = TP_k / (TP_k + FP_k), \quad Recall_k = TP_k / (TP_k + FN_k), \quad F1_k = 2 \cdot Precision_k \cdot Recall_k / (Precision_k + Recall_k).$$

Macro averaging takes the unweighted mean of the per-class values, so that the 35 Pathologic test cases carry the same weight as the 332 Normal cases. Overall accuracy is the fraction of correct predictions. The **ROC-AUC** is computed one-vs-rest (OvR) for each class from predicted probabilities and macro-averaged; it equals the probability that a random positive example receives a higher score than a random negative one, and is independent of any single decision threshold.

For regression with targets y_i, predictions $\hat{y}_i$ and test mean $\bar{y}$:

$$MSE = (1/n)\,\Sigma\,(y_i - \hat{y}_i)^2, \quad RMSE = \sqrt{MSE}, \quad R^2 = 1 - \Sigma\,(y_i - \hat{y}_i)^2 \,/\, \Sigma\,(y_i - \bar{y})^2.$$

R^2 is negative whenever the model's squared error exceeds that of always predicting the test mean. For clustering we use the within-cluster sum of squares (WCSS), the mean silhouette coefficient $s = (b - a)/\max(a, b)$ (where a is the mean intra-cluster distance and b the mean distance to the nearest other cluster), and the Adjusted Rand Index (ARI), which equals 1 for identical partitions and about 0 for random agreement. For PCA we use the explained variance ratio $r_j = \lambda_j / \Sigma\,\lambda_m$ and the reconstruction mean-squared error of the rank-k projection.

1.5 Notation

Symbol	Meaning	Symbol	Meaning
n, p	Number of samples, number of features	η	Gradient-descent learning rate
K	Number of clusters (K-Means) or classes (classification)	α	Regularisation strength (Ridge, Lasso)
k	Neighbours in KNN; number of retained principal components	C, γ	SVM penalty and RBF kernel width
w, b	Regression weights and intercept	λ_j, v_j	j-th eigenvalue and eigenvector of the covariance matrix
$\hat{y}$, $\bar{y}$	Prediction; mean of the target	μ_k	Centroid of cluster k
Σ	Covariance matrix (Section 4.4)	r_j	Explained variance ratio of component j

2. Q1 — Multiclass Classification on Cardiotocography

2.1 Dataset Description, Preprocessing, and Anti-Leakage Safeguards

The Cardiotocography dataset contains 2,126 recordings extracted from fetal cardiotocograms. Each record comprises 21 numeric features capturing diagnostic metrics such as baseline heart rate (`LB`), accelerations (`AC`), fetal movements (`FM`), uterine contractions (`UC`), decelerations (`DL`, `DS`, `DP`), and short/long-term heart rate variability indices (`ASTV`, `MSTV`, `ALTV`, `MLTV`), as well as histogram summary statistics (`Width`, `Min`, `Max`, `Nmax`, `Nzeros`, `Mode`, `Mean`, `Median`, `Variance`, `Tendency`). There are no missing values; 15 duplicate rows exist and were kept (reported for transparency). Table 2.1 lists the feature scales, which differ by several orders of magnitude (for example `DS` is of order 10^{-4} while `Max` is of order 10^{2}) and therefore motivate standardisation for distance-based and kernel-based models.

Table 2.1: Summary Statistics of the 21 CTG Input Features (N = 2,126)

Feature	Mean	Std	Min	Median	Max
LB	133.304	9.841	106.0	133.000	160.000
AC	0.003	0.004	0.0	0.002	0.019
FM	0.009	0.047	0.0	0.000	0.481
UC	0.004	0.003	0.0	0.004	0.015
DL	0.002	0.003	0.0	0.000	0.015
DS	0.000	0.000	0.0	0.000	0.001
DP	0.000	0.001	0.0	0.000	0.005
ASTV	46.990	17.193	12.0	49.000	87.000
MSTV	1.333	0.883	0.2	1.200	7.000
ALTV	9.847	18.397	0.0	0.000	91.000
MLTV	8.188	5.628	0.0	7.400	50.700
Width	70.446	38.956	3.0	67.500	180.000
Min	93.579	29.560	50.0	93.000	159.000
Max	164.025	17.944	122.0	162.000	238.000
Nmax	4.068	2.949	0.0	3.000	18.000
Nzeros	0.324	0.706	0.0	0.000	10.000
Mode	137.452	16.381	60.0	139.000	187.000
Mean	134.611	15.594	73.0	136.000	182.000
Median	138.090	14.467	77.0	139.000	186.000
Variance	18.808	28.978	0.0	7.000	269.000
Tendency	0.320	0.611	-1.0	0.000	1.000

Target formulation and leakage prevention. The dataset contains two potential target fields: `NSP` (a 3-class diagnosis: 1 = Normal, 2 = Suspect, 3 = Pathologic) and `CLASS` (a 10-class morphologic pattern code). Because `CLASS` is a granular derivative of the clinical diagnosis, retaining `CLASS` as an input feature would constitute label leakage. Consequently `CLASS` was strictly excluded and `NSP` was label-encoded to indices {0, 1, 2}. Among the inputs, only the histogram statistics `Mode`–`Median` ($r = 0.93$) and `Mean`–`Median` ($r = 0.95$) exceed $|r| = 0.9$, so the feature set is only mildly redundant.

Stratified partitioning. The data are split into an 80% training set (1,700 samples) and a 20% test set (426 samples) using stratified sampling (`random_state=42`). Stratification is essential because Pathologic samples constitute only 176 instances in the entire corpus; an unstratified split risks high variance in the minority-class test counts. The training partition contains 1,323 Normal (77.8%), 236 Suspect (13.9%), and 141 Pathologic (8.3%) instances; the test partition preserves 332 Normal (77.9%), 59 Suspect (13.8%), and 35 Pathologic (8.2%) instances, summing to the full 2,126 recordings.

Pipeline encapsulation. To eliminate validation peeking, `StandardScaler` transformations are embedded within `sklearn.pipeline.Pipeline` objects, ensuring scalers are fitted solely on training folds during 5-fold cross-validation.

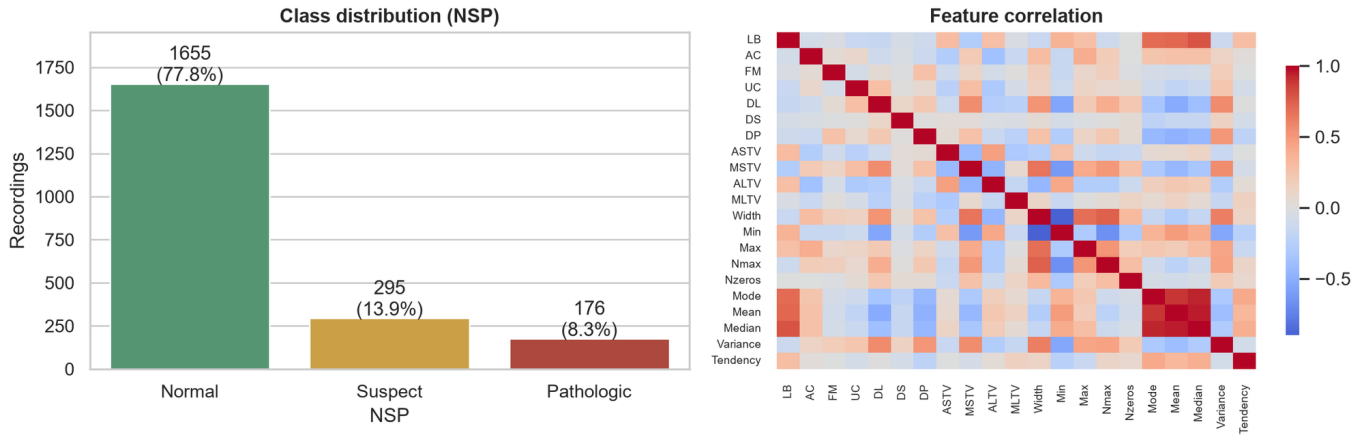


Figure 2.1: Exploratory Data Analysis — (Left) class distribution of NSP showing the pronounced 78/14/8 imbalance; (Right) Pearson correlation matrix showing collinear histogram clusters.

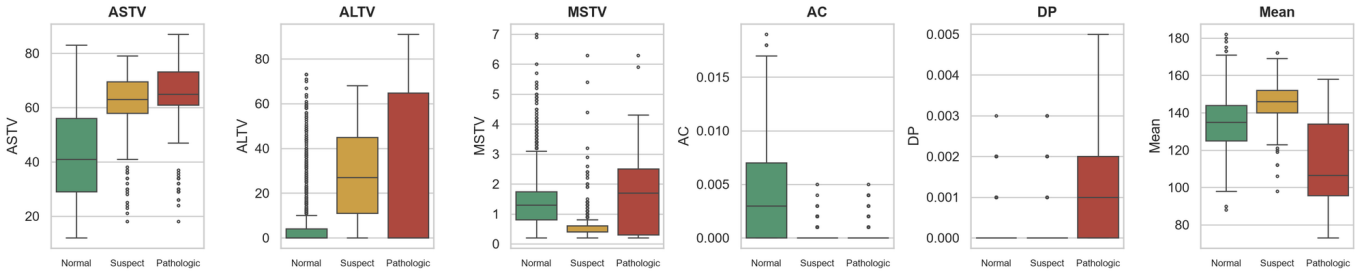


Figure 2.2: Boxplots of key predictive features (`ASTV`, `ALTV`, `MSTV`, `AC`, `DP`, `Mean`) segregated across Normal, Suspect, and Pathologic classes.

Figure 2.2 shows why the problem is learnable: ASTV (percentage of time with abnormal short-term variability) rises from Normal to Pathologic, ALTV is near zero for Normal recordings but widely spread for Suspect and Pathologic ones, and AC (accelerations) is essentially absent in the abnormal classes. The overlap between Suspect and the other two classes in most features also foreshadows that Suspect will be the hardest class.

2.2 Classifiers, Hyper-parameter Optimization and Model Architecture

Four model families were benchmarked: (a) K-Nearest Neighbours, (b) Decision Trees, (c) Support Vector Machines with RBF kernel, and (d) a deep 1D Convolutional Neural Network (bonus). All classical models were tuned by systematic 5-fold stratified cross-validation optimizing **macro-F1** (`f1_macro`), which gives equal weight to the minority classes. Table 2.2 summarises the search spaces and the selected configurations.

Table 2.2: Hyper-parameter Search Spaces and Selected Configurations (Q1)

Model	Search space	Selected configuration	CV macro-F1
KNN	k ∈ [1, 30]; weights ∈ {uniform, distance}; metric ∈ {euclidean, manhattan}	k = 4, Manhattan, distance weighting	0.8541
Decision Tree	max_depth ∈ {2…15, None}; criterion ∈ {gini, entropy}; min_samples_leaf ∈ [1, 10]; ccp_alpha ∈ [0.0, 0.005]	entropy, max_depth = 8, min_samples_leaf = 1, ccp_alpha = 0.0	0.8900
SVM (RBF)	C ∈ {0.1, 0.5, 1, 5, 10, 50, 100, 500}; γ ∈ {0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.0}	C = 100, γ = 0.03	0.8759
1D-CNN	Fixed architecture; AdamW (weight decay 1e-4); ReduceLROnPlateau; early stopping (patience 25) on 15% internal validation split	105,123 trainable parameters; 5 seeds (42–46)	n/a (validation loss)

K-Nearest Neighbours

KNN classifies a point by a vote among its k nearest training points. With distance weighting, neighbour i contributes a vote of weight $1/d_i$, so the prediction is

$$\hat{y}(x) = \text{argmax}_c \sum_{i \in N_k(x)} 1[y_i = c] / d(x, x_i).$$

KNN has no training phase, and its complexity is controlled by k: small k gives a flexible, high-variance boundary, large k a smooth, high-bias one. Optimal parameters were **k = 4, metric = Manhattan, weights = distance**. With distance weighting a training point is its own nearest neighbour at distance zero (weight → ∞), so evaluating training samples against themselves

yields an empirical training score of 1.0 for all values of *k*; the meaningful complexity signal is therefore the validation curve in Figure 2.6.

Decision Tree

A decision tree recursively partitions the feature space with axis-aligned thresholds chosen to maximise information gain, i.e. the reduction of the entropy $H = -\Sigma_c\ p_c\ \log_2\ p_c$. Complexity is controlled by `max_depth`, leaf size and cost-complexity pruning `ccp_alpha`. The optimal tree uses **criterion = entropy, max_depth = 8, min_samples_leaf = 1, ccp_alpha = 0.0** (CV macro-F1 0.8900). An unpruned tree grows to depth 14 with 103 leaves and reaches 99.94% train accuracy but only 90.14% test accuracy, a classic overfitting signature.

Support Vector Machine with RBF kernel

The soft-margin SVM maximises the margin while penalising violations with weight *C*. The RBF kernel $k(x, x') = \exp(-\gamma\|x - x'\|^2)$ implicitly maps the data to an infinite-dimensional space; γ sets the kernel width (large γ = very local, flexible boundary) and *C* the penalty for training errors (large *C* = low-bias, high-variance). The multiclass problem is handled by scikit-learn's one-vs-one scheme, and probability estimates are produced by internal Platt scaling. The optimum was **C = 100, γ = 0.03** (CV macro-F1 0.8759). As an additional check, setting `class_weight='balanced'` lowered the test macro-F1 from 0.8450 to 0.8180 and left the Pathologic recall unchanged at 0.8571, so the tuned unweighted SVM was retained.

Deep 1D Convolutional Neural Network (bonus)

CTG data are tabular and lack intrinsic spatial ordering. To evaluate deep representations regardless, we designed an 8-layer network (six convolutional and two dense layers): `Input(1, 21)` → [Conv1D(1→32, k=3, p=1) → BatchNorm → ReLU → Conv1D(32→32) → BN → ReLU → MaxPool(2)] → [Conv1D(32→64) → BN → ReLU → Conv1D(64→64) → BN → ReLU → MaxPool(2)] → [Conv1D(64→128) → BN → ReLU → Conv1D(128→128) → BN → ReLU] → AdaptiveAvgPool1D(1) → Flatten → Dropout(0.3) → Dense(64) → ReLU → Dropout(0.3) → Dense(3). The network has **105,123** trainable parameters. Regularization uses weight decay (1e-4) with the AdamW optimizer, a `ReduceLROnPlateau` learning-rate scheduler, dropout, batch normalisation, and early stopping (patience = 25 epochs) on an internal 15% validation split (hence 1,445 training samples and 255 validation samples). Severe class imbalance is mitigated with inverse-frequency class weights in the cross-entropy loss, $L = -\Sigma_i\ w_{y_i}\ \log\ p(y_i \mid x_i)$ with $w_c \propto 1/n_c$.

2.3 Benchmark Performance and Confusion Matrix Analysis

All four classifiers were evaluated on the held-out 426-instance test set. Table 2.3 presents the test metrics.

Table 2.3: Q1 Classifier Evaluation on the Held-Out Test Set (N = 426)

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)	ROC-AUC (OvR Macro)
KNN (k=4, Manhattan, Dist)	0.9108	0.8328	0.8186	0.8255	0.9531
Decision Tree (Depth=8)	0.9155	0.8583	0.8113	0.8321	0.9171
SVM (RBF) (C=100, γ=0.03)	**0.9178**	**0.8649**	0.8293	**0.8450**	**0.9696**
1D CNN (Bonus) (Seed 42)	0.8826	0.7538	**0.8677**	0.7981	0.9564

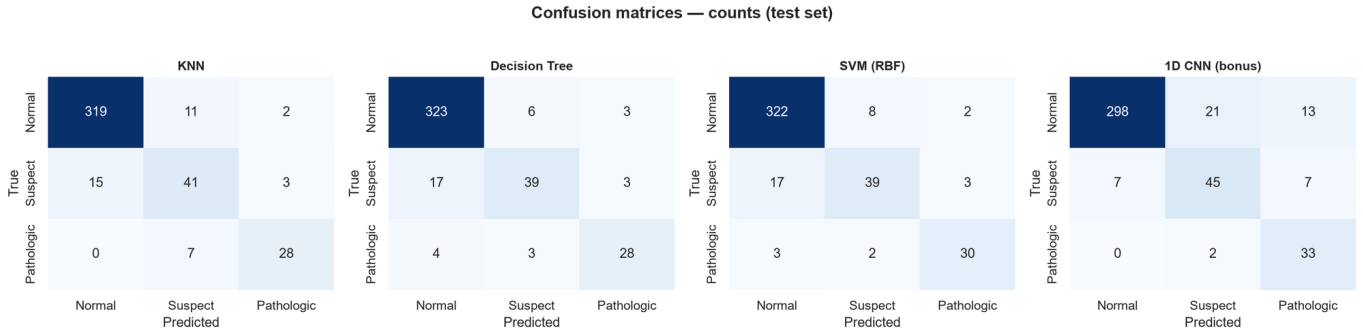


Figure 2.3: Absolute confusion matrices on the held-out test set (rows: true class; columns: predicted class).

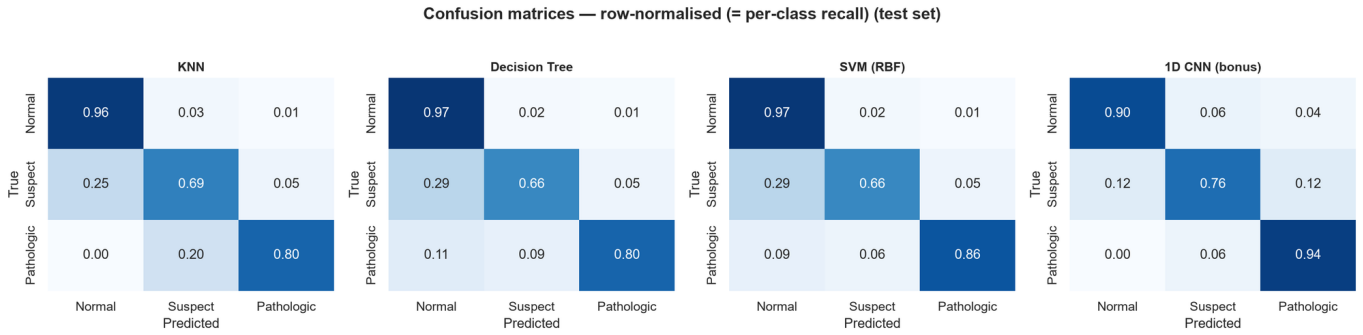


Figure 2.4: Row-normalised confusion matrices (equivalent to per-class recall / sensitivity).

Clinical confusion analysis. In obstetrics the critical error is classifying a Pathologic fetus as Normal. Examining the confusion matrices (Figure 2.3 counts, Figure 2.4 row-normalised recalls):

- **SVM (RBF)** correctly identifies 322 of 332 Normal (96.99% recall), 39 of 59 Suspect (66.10% recall), and 30 of 35 Pathologic fetuses (85.71% recall). Only 3 Pathologic cases are misdiagnosed as Normal (8.57%) and 2 as Suspect (5.71%).
- **1D-CNN**, guided by inverse-frequency loss weighting, achieves the highest Pathologic recall at 94.29% (33 of 35 identified), with 0 misdiagnosed as Normal and 2 as Suspect (5.71%). This high sensitivity reduces Normal recall to 89.76% (298 of 332 correct), misclassifying 34 Normal cases as Suspect or Pathologic.
- **Decision Tree** identifies 323 of 332 Normal (97.29%), 39 of 59 Suspect (66.10%), and 28 of 35 Pathologic fetuses (80.00% recall), missing 4 Pathologic cases as Normal (11.43%) and 3 as Suspect.
- **KNN** correctly classifies 319 of 332 Normal (96.08%), 41 of 59 Suspect (69.49%), and 28 of 35 Pathologic fetuses (80.00% recall), misdiagnosing 0 Pathologic cases as Normal and 7 as Suspect.

KNN and the CNN made no Pathologic→Normal errors, whereas the SVM made 3. However, KNN missed 7 Pathologic cases as Suspect, and the CNN's 94% Pathologic recall came at the cost of Normal recall (89.8%) and of Pathologic precision (0.623, see Table 2.4). We select SVM-RBF because it has the highest macro and Pathologic ROC-AUC with a smaller train–test gap, although its AUC advantage over KNN and the CNN is not statistically significant. If missed Pathologic cases are the overriding priority, the CNN is the better operating point.

Table 2.4: Per-Class Precision, Recall and F1-score on the Test Set (derived from the confusion matrices in Figure 2.3)

Model	Class	Precision	Recall	F1-score	Support
KNN	Normal	0.955	0.961	0.958	332
	Suspect	0.695	0.695	0.695	59
	Pathologic	0.848	0.800	0.824	35
Decision Tree	Normal	0.939	0.973	0.956	332
	Suspect	0.812	0.661	0.729	59
	Pathologic	0.824	0.800	0.812	35
SVM (RBF)	Normal	0.942	0.970	0.955	332
	Suspect	0.796	0.661	0.722	59
	Pathologic	0.857	0.857	0.857	35
1D CNN (bonus)	Normal	0.977	0.898	0.936	332
	Suspect	0.662	0.763	0.709	59
	Pathologic	0.623	0.943	0.750	35

Table 2.4 makes the trade-offs explicit. All models classify Normal recordings very well (F1 ≥ 0.936). Suspect is the hardest class for every model (F1 between 0.695 and 0.729) because it lies between Normal and Pathologic in feature space. For Pathologic, the SVM balances precision and recall (both 0.857), whereas the CNN trades precision (0.623) for recall (0.943).

2.4 Receiver Operating Characteristic (ROC) and Bootstrap Significance Analysis

Because accuracy is distorted by the 78% dominant Normal class, One-vs-Rest (OvR) ROC curves provide a threshold-independent measure of discrimination. Figure 2.5 depicts the three one-vs-rest curves and the macro-average curve (the original four-panel figure is shown in two rows for legibility).

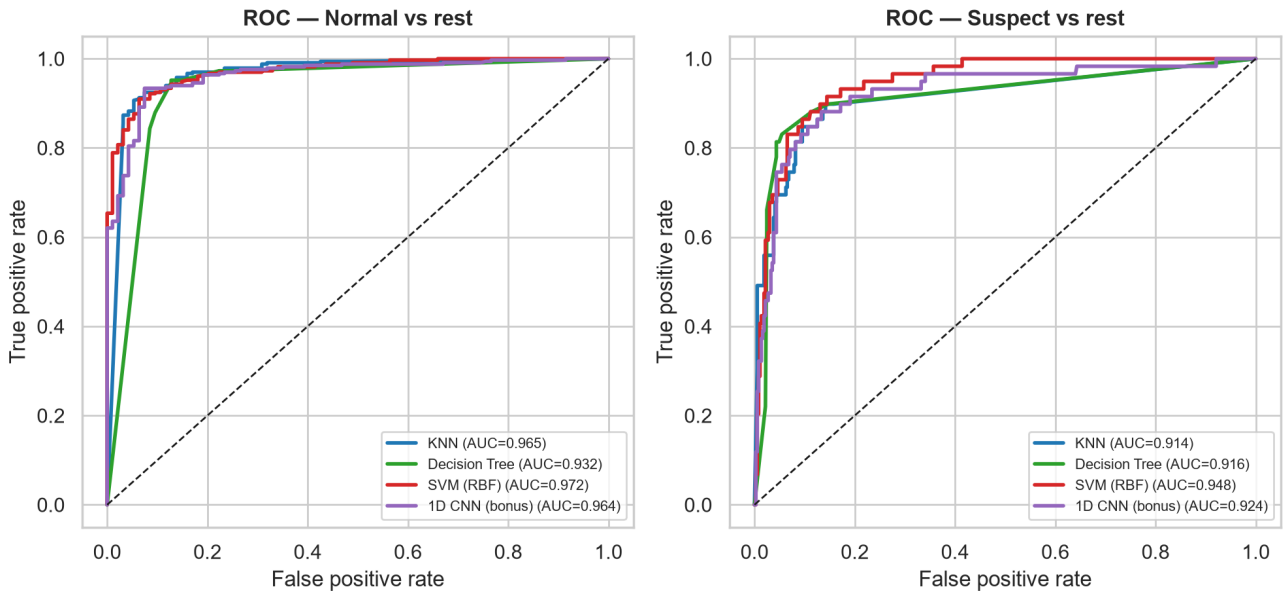


Figure 2.5 (a): One-vs-Rest ROC curves for the Normal and Suspect classes on the held-out test set.

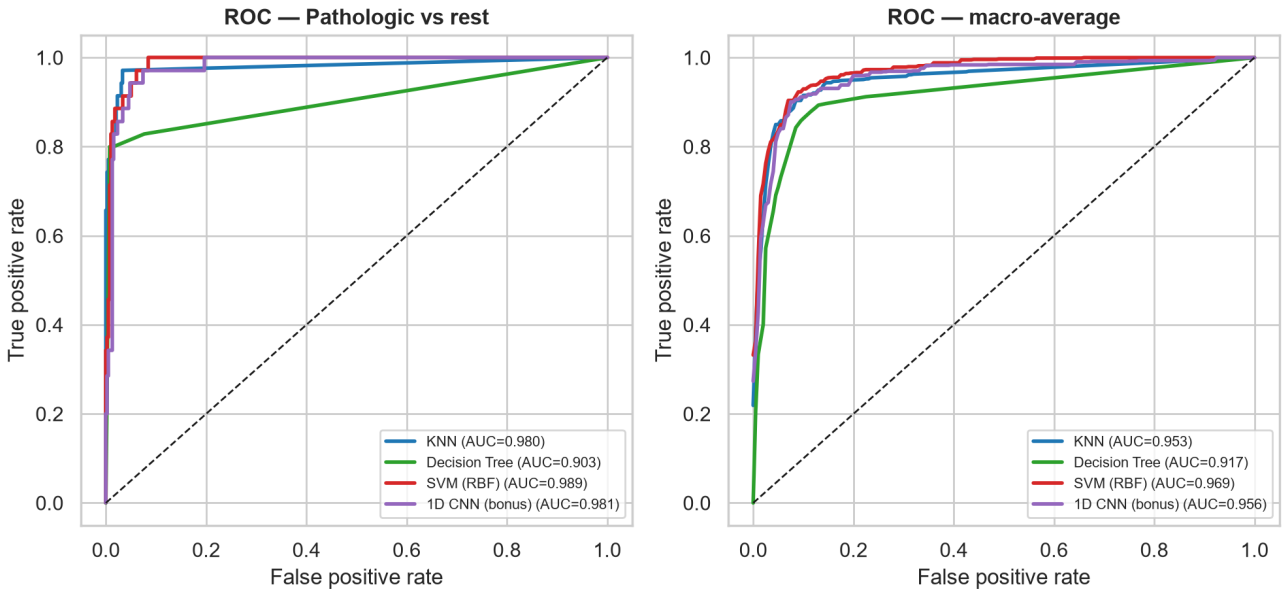


Figure 2.5 (b): One-vs-Rest ROC curve for the Pathologic class and the macro-average ROC curve.

Table 2.5: Per-Class ROC-AUC and 500-Iteration Bootstrap 95% Confidence Intervals

Model	Normal AUC	Suspect AUC	Pathologic AUC	Macro AUC	95% Bootstrap CI	ΔAUC vs SVM [95% CI]
SVM (RBF)	0.9717	0.9481	**0.9889**	**0.9696**	[0.9546, 0.9814]	0.0000 (Reference)
1D CNN (bonus)	0.9636	0.9244	0.9811	0.9564	[0.9356, 0.9731]	−0.0130 [−0.0293, +0.0009]
KNN	0.9651	0.9137	0.9805	0.9531	[0.9289, 0.9744]	−0.0166 [−0.0348, +0.0007]
Decision Tree	0.9319	0.9161	0.9033	0.9171	[0.8747, 0.9501]	**−0.0536 [−0.0906, −0.0220]**

Determination of the best classifier

1. **Headline ROC ranking.** SVM (RBF) attains the highest macro-average ROC-AUC of **0.9696**, followed by the 1D-CNN (0.9564), KNN (0.9531), and the Decision Tree (0.9171).

2. **Critical minority detection.** For the clinically vital Pathologic category, SVM-RBF achieves an OvR AUC of **0.9889**, higher than the CNN (0.9811), KNN (0.9805), and the Decision Tree (0.9033).

3. **Bootstrap significance test.** Because the test set contains only 35 Pathologic instances, we evaluated 500 paired bootstrap iterations. The 95% paired confidence interval for $\Delta AUC = AUC_{Model} - AUC_{SVM}$ contains zero for both the 1D-CNN ([−0.0293, +0.0009]) and KNN ([−0.0348, +0.0007]). Thus the difference in ROC-AUC between SVM, CNN, and KNN is **not statistically**

significant (borderline, as both paired intervals end just above zero). Only the Decision Tree is significantly inferior (paired interval [−0.0906, −0.0220], excluding zero).

4. **CNN multi-seed stability.** Evaluating the 1D-CNN across 5 random seeds (Table 2.6; `results/q1_cnn_seed_stability.csv`) yields (mean ± std) accuracy 0.8789 ± 0.0103, macro precision 0.7622 ± 0.0175, macro recall 0.8567 ± 0.0141, macro-F1 0.7991 ± 0.0117, macro ROC-AUC 0.9628 ± 0.0044, and 35.2 ± 3.7 epochs trained. The seed-to-seed spread is small relative to the gap to the SVM in macro-F1, so the CNN conclusion is not an artefact of one lucky or unlucky seed.

Table 2.6: 1D-CNN Stability Across Five Random Seeds (Test Set)

Seed	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	ROC-AUC (OvR macro)	Epochs	Best val. loss
42	0.8826	0.7538	0.8677	0.7981	0.9564	41	0.3026
43	0.8709	0.7805	0.8387	0.7989	0.9654	31	0.2610
44	0.8920	0.7752	0.8725	0.8161	0.9648	34	0.3204
45	0.8662	0.7369	0.8576	0.7831	0.9604	36	0.2718
46	0.8826	0.7645	0.8468	0.7994	0.9671	34	0.2735
Mean ± std	**0.8789 ± 0.0103**	**0.7622 ± 0.0175**	**0.8567 ± 0.0141**	**0.7991 ± 0.0117**	**0.9628 ± 0.0044**	**35.2 ± 3.7**	**—**

2.5 ROC-Based Conclusion

Taking the ROC analysis together, **SVM with RBF kernel is the best classifier** among those compared: it has the highest macro AUC and the highest Pathologic AUC, the smallest AUC gap between train and test (0.0285, Table 2.7), and its bootstrap interval is the tightest of the four. The honest qualification is that the advantage over KNN and the CNN is a point estimate that is not statistically significant on a test set with only 35 Pathologic cases, whereas the Decision Tree is demonstrably worse.

2.6 Overfitting and Model Complexity Analysis

To evaluate capacity control and generalisability we analysed the divergence between training and test performance (Table 2.7) and the validation behaviour as a function of model complexity (Figures 2.6–2.9).

Table 2.7: Overfitting Gap Analysis (Train vs. Test Performance)

Model	Train Acc	Test Acc	Accuracy Gap	Train Macro F1	Test Macro F1	F1-Score Gap	ROC-AUC Gap
KNN (k=4, distance)	0.9994	0.9108	0.0886	0.9992	0.8255	0.1737	0.0469
Decision Tree (Depth=8)	0.9800	0.9155	0.0645	0.9682	0.8321	0.1362	0.0813
SVM (RBF) (C=100)	0.9806	0.9178	**0.0627**	0.9684	0.8450	**0.1234**	**0.0285**
1D CNN (Bonus)	0.9594	0.8826	0.0768	0.9274	0.7981	0.1293	0.0409

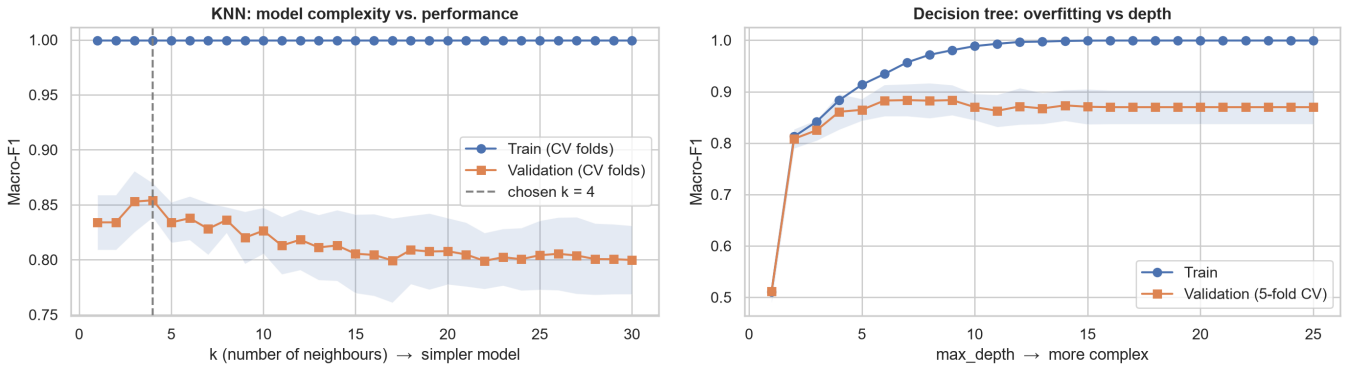


Figure 2.6: Model complexity curves — (Left) KNN performance vs k, showing underfitting at large k; (Right) Decision Tree performance vs maximum depth, showing a widening generalisation gap.

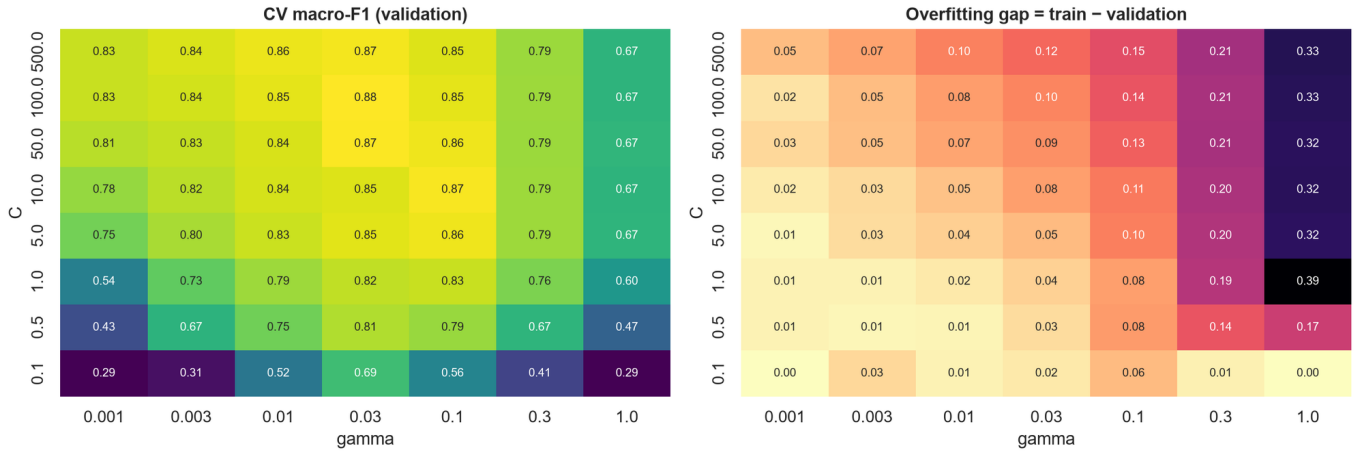


Figure 2.7: SVM grid-search heatmaps — (Left) 5-fold CV validation macro-F1 across C and γ; (Right) overfitting gap (train F1 − validation F1), exposing severe overfit in the high-C, high-γ corner.

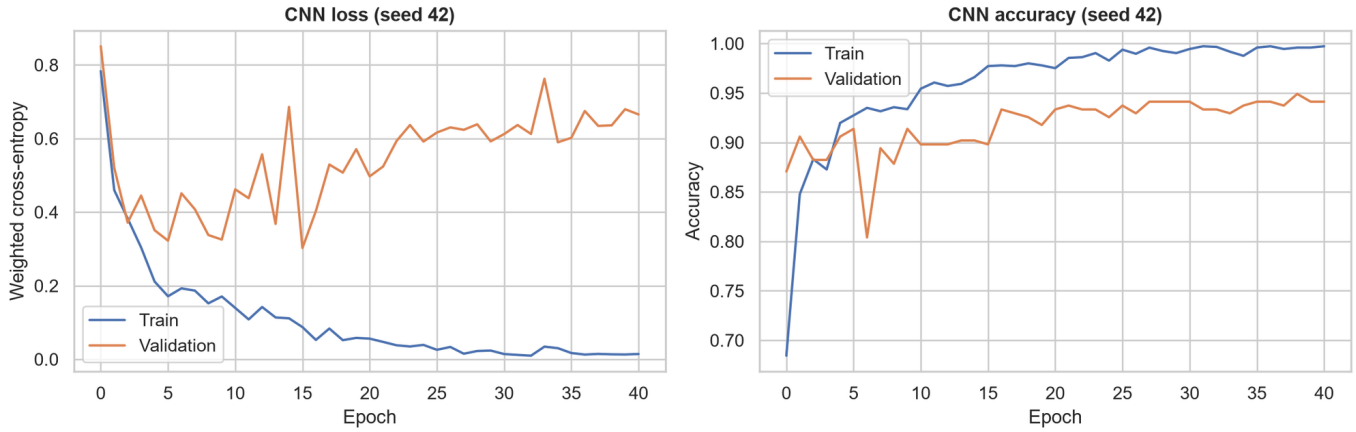


Figure 2.8: 1D-CNN training curves (seed 42) — weighted cross-entropy loss and accuracy across 41 epochs before early stopping triggered.

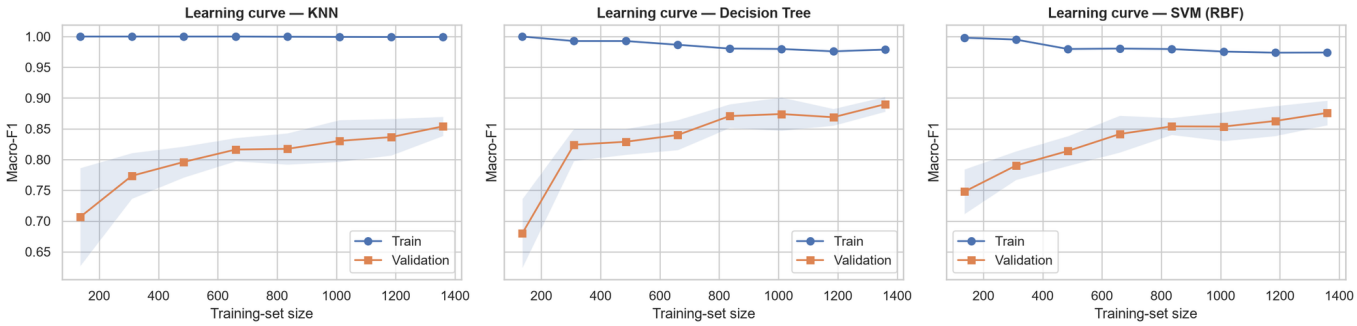


Figure 2.9: Empirical learning curves for KNN, Decision Tree, and SVM-RBF evaluating training-set size scaling.

Synthesis of complexity dynamics

- **KNN.** Distance weighting gives an exact training score of 1.0 (train F1 = 0.9992) for all *k*. As k grows beyond about 10 the validation macro-F1 steadily degrades toward underfitting (Figure 2.6, left), and the best validation score occurs at the small value k = 4. The largest train–test F1 gap (0.1737) of all models reflects its memorising nature.
- **Decision Tree.** An unconstrained tree splits down to single-instance leaves. The training curve saturates near 1.0 beyond depth 10 while validation stalls near 0.87–0.88, and pruning to depth 8 balances complexity, although axis-aligned thresholds remain vulnerable to oblique class boundaries. Its AUC gap (0.0813) is the largest, consistent with its poorer probability estimates (a tree outputs piecewise-constant leaf proportions).
- **SVM-RBF.** Figure 2.7 illustrates both bias–variance axes: the lowest CV F1 (0.29) occurs at C = 0.1, γ = 0.001 (underfitting: too smooth and too tolerant), the largest train–validation gap (0.39) at C = 1, γ = 1.0 and 0.33 at C = 500, γ = 1.0 (overfitting: too local). The optimal zone at C = 100, γ = 0.03 achieves a test macro-F1 of 0.8450 with an F1 generalisation gap of 0.1234, the smallest of the four models.
- **1D-CNN vs. tabular inductive bias.** The CNN achieved competitive accuracy (0.8826) and recall (0.8677) but required extensive regularisation (dropout, batch normalisation, weight decay, early stopping) to prevent 105,123 parameters from

memorising 1,445 training samples. Validation loss plateaus and begins rising after approximately epoch 15 (seed 42 reaches its minimum validation loss of 0.3026 at epoch 16 and rises thereafter while the training loss drops toward zero; with patience = 25, training stopped at epoch 41), demonstrating clear overfitting after epoch 16, limited by early stopping with best-weight restoration. Crucially, CNNs assume translation invariance and local feature adjacency (adjacent pixels or audio samples). In tabular CTG data the feature ordering is arbitrary: LB is not spatially adjacent to AC. Forcing 1D convolutions across tabular columns imposes an artificial inductive bias, and the CNN did not outperform the RBF-SVM.

Learning curves (Figure 2.9). For all three classical models the validation curve is still rising at the full training size (about 1,360 samples in the cross-validation folds), while the training curve stays near the top. This indicates that the models are limited by variance rather than bias; more labelled CTG recordings, especially Pathologic ones, would be the most effective improvement.

2.7 Summary of Q1 Findings

Assignment item	Finding
(i) 80/20 split	Stratified split: 1,700 training and 426 test recordings; class proportions preserved (Normal 77.9%, Suspect 13.8%, Pathologic 8.2% in the test set).
(ii) Confusion matrices	Figures 2.3 and 2.4. Pathologic→Normal errors: KNN 0, CNN 0, SVM 3, Decision Tree 4 (of 35 Pathologic cases).
(iii) Metrics	SVM: best accuracy (0.9178), macro precision (0.8649), macro-F1 (0.8450) and macro ROC-AUC (0.9696). CNN: best macro recall (0.8677). Tables 2.3–2.5.
(iv) Overfitting / complexity	Unpruned tree 99.94% train vs. 90.14% test accuracy; KNN train score 1.0; SVM has the smallest train–test gaps; CNN overfits after epoch 16 without early stopping.
(v) Best by ROC	**SVM-RBF** (macro AUC 0.9696; Pathologic AUC 0.9889); difference to KNN and CNN not significant, Decision Tree significantly inferior.

3. Q2 — Continuous Regression on Parkinson's Telemonitoring

3.1 Dataset Architecture, Subject Disjointness, and Data Leakage

The Parkinson's Telemonitoring dataset (UCI ID 189) comprises 5,875 recordings gathered from **42 individuals** with early-stage Parkinson's disease, monitored remotely over a 6-month trial via an at-home telemonitoring device. The feature space consists of 19 numeric predictors: demographic variables (`age`, `sex`), temporal trajectory (`test_time`), and 16 clinical voice measures quantifying vocal frequency perturbation (`Jitter(%)`, `Jitter(Abs)`, `Jitter:RAP`, `Jitter:PPQ5`, `Jitter:DDP`), amplitude perturbation (`Shimmer`, `Shimmer(dB)`, `Shimmer:APQ3`, `Shimmer:APQ5`, `Shimmer:APQ11`, `Shimmer:DDA`), noise ratios (`NHR`, `HNR`), and non-linear dynamic metrics (`RPDE`, `DFA`, `PPE`). There are no missing values.

Sibling target and ID exclusion. The clinical rating scale includes both `motor_UPDRS` and `total_UPDRS`. Because `motor_UPDRS` is a sub-component of the total score with strong clinical collinearity, its inclusion would constitute direct label leakage. Both `motor_UPDRS` and the patient identifier `subject#` were removed from the input matrix.

The subject-wise evaluation mandate. Each patient contributes approximately 140 recordings. The target is dominated by between-patient differences: the mean within-patient variance of `total_UPDRS` is only 7.9, whereas the variance of the patient means is 109.1, about 14 times larger. Because voice acoustics, vocal anatomy, age and biological baseline are patient-specific, recordings of the same individual are highly correlated. If a standard random train/test split is applied, recordings from the same patient appear in both partitions (in our random-row split, all 42 patients are present in both train and test), and the model trivially memorises patient identities rather than learning generalizable vocal biomarkers of disease severity. To enforce clinical validity we implement `GroupShuffleSplit` (80/20 subject-wise split): 33 patients (4,640 records) form the training set and 9 completely unseen patients (1,235 records) form the held-out test set (`assert set(gtr).isdisjoint(gte)` verified). All hyper-parameter tuning is restricted to training patients via 5-fold `GroupKFold`.

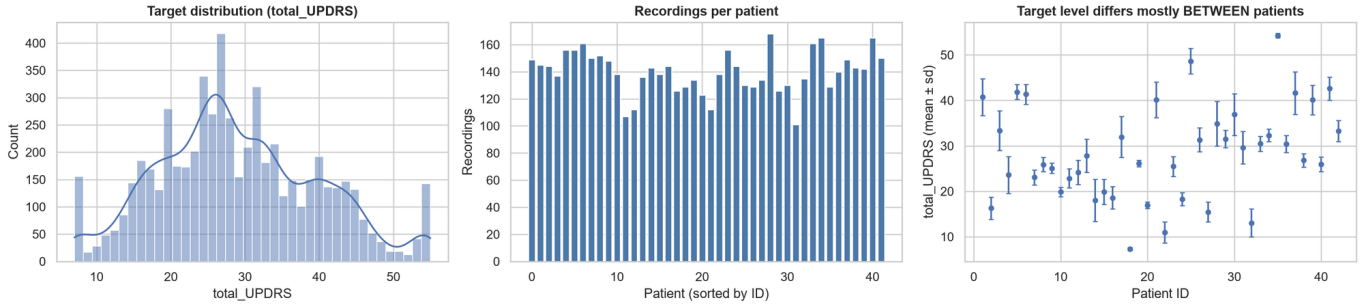


Figure 3.1: (Left) Distribution of `total_UPDRS`; (Middle) recording counts per patient; (Right) patient mean total UPDRS ± standard deviation, showing substantial between-patient variation.

Table 3.1: The Data Leakage Experiment — Subject-Wise vs. Random-Row Splitting

Model	Evaluation split strategy	Test MSE	Test RMSE	Test R^2	Test Pearson R
Ridge (CV α)	Subject-wise (honest, unseen patients)	99.9557	9.9978	−0.2063	0.1359
Ridge (CV α)	Random rows (leaky, patients in both)	95.2589	9.7601	0.1404	0.3945
Lasso (CV α)	Subject-wise (honest, unseen patients)	106.0178	10.2965	−0.2795	0.1005
Lasso (CV α)	Random rows (leaky, patients in both)	100.3579	10.0179	0.0944	0.3287
Polynomial Deg 1 (Top-5)	Subject-wise (honest, unseen patients)	130.4986	11.4236	−0.5749	0.1044
Polynomial Deg 1 (Top-5)	Random rows (leaky, patients in both)	98.1906	9.9091	0.1139	0.3402
Random Forest (non-linear extra)	Subject-wise (honest, unseen patients)	179.4425	13.3956	−1.1656	−0.0345
Random Forest (non-linear extra)	Random rows (leaky, patients in both)	2.5000	1.5812	**+0.9774**	0.9891

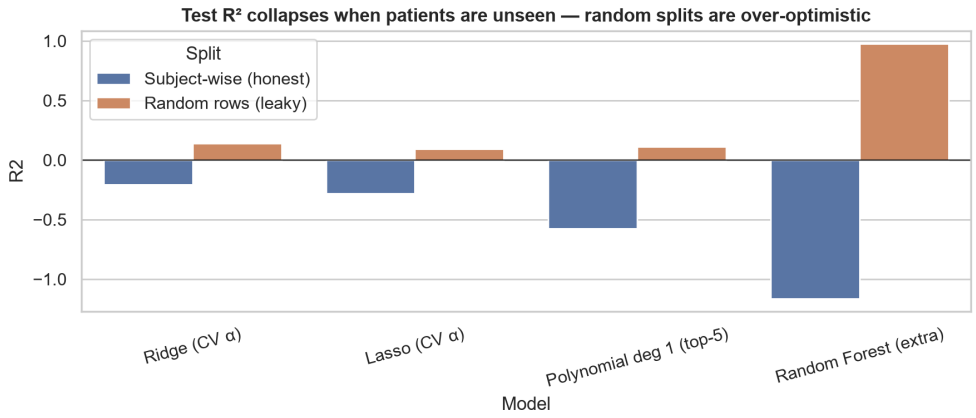


Figure 3.2: Test R^2 comparison — honest subject-wise evaluation vs. leaky random-row splitting across models.

Empirical proof of leakage. Table 3.1 and Figure 3.2 demonstrate the leakage phenomenon. A highly expressive non-linear model (Random Forest) achieves a near-perfect R^2 of **+0.9774** (RMSE 1.58) under random splitting because it can partition the feature space to memorise patient vocal signatures. On unseen subjects its performance collapses to R^2 = −1.1656 (RMSE 13.40), far worse than predicting the training mean. The high R^2 values obtainable on this dataset with random splits are therefore artefacts of leaked patient identity. Even the simple linear models are affected: Ridge changes from R^2 = +0.1404 to −0.2063 when the evaluation becomes patient-disjoint.

3.2 Multicollinearity and Near-Exact Linear Dependencies

Voice perturbation measures are mathematically related by definition. An audit reveals near-exact linear dependencies in the data:

- `Jitter:DDP` ≈ 3.0 × `Jitter:RAP` (ratio mean = 3.0001, std = 0.0047);
- `Shimmer:DDA` ≈ 3.0 × `Shimmer:APQ3` (ratio mean = 3.0000, std = 0.0008).

The tiny deviations arise from rounding in the stored values; for modelling purposes each pair is perfectly collinear. Variance Inflation Factors (VIF) computed on standardised features confirm extreme collinearity (Table 3.2): `Shimmer:APQ3`, `Jitter:DDP`, `Jitter:RAP` and `Shimmer:DDA` all hit the numerical ceiling of VIF = 1,000,000, and 10 of the 19 features exceed the usual threshold of 10.

Table 3.2: Variance Inflation Factors of the Most Collinear Predictors (standardised features)

Feature	VIF	Feature	VIF
`Shimmer:APQ3`	1,000,000 (ceiling)	`Shimmer:APQ5`	52.6
`Jitter:DDP`	1,000,000 (ceiling)	`Jitter:PPQ5`	31.0
`Jitter:RAP`	1,000,000 (ceiling)	`Shimmer:APQ11`	15.3
`Shimmer:DDA`	1,000,000 (ceiling)	`NHR`	8.6
`Shimmer`	173.5	`Jitter(Abs)`	7.9
`Jitter(%)`	89.0	`Shimmer(dB)`	76.9

Under ordinary least squares (OLS) the normal equation $w = (X^TX)^{-1}X^Ty$ involves an ill-conditioned Gram matrix and produces wild, unstable coefficients (from `q2_coefficients.csv`: `Jitter:RAP` = −166.6397 vs. `Jitter:DDP` = +169.7507, and `Shimmer:APQ3` = −783.7155 vs. `Shimmer:DDA` = +779.5799, alongside `age` = +3.9923 and `PPE` = +2.2992). Large positive and negative coefficients of near-identical features cancel each other on the training data, which is exactly the high-variance behaviour that regularisation is designed to suppress.

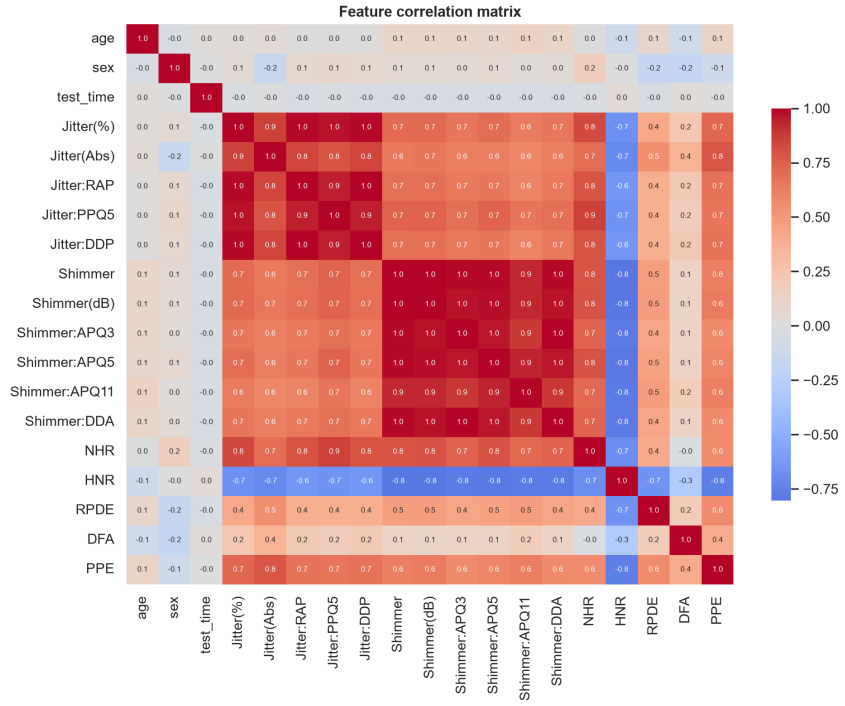


Figure 3.3: Correlation heatmap of the 19 telemonitoring predictors, highlighting severe collinear blocks among the jitter and shimmer variants.

3.3 Linear Regression with Gradient Descent from Scratch

Mathematical formulation. We formulate linear regression as $\hat{y} = Xw + b$ and minimise the mean squared error loss with a halving factor for gradient simplicity:

$$J(w, b) = (1/2n)\, \Sigma_{i=1..n}\, (\hat{y}_i - y_i)^2 = (1/2n)\, \|Xw + b\cdot 1 - y\|^2.$$

The analytical gradients with respect to the weights and the intercept are

$$\partial J/\partial w = (1/n)\, X^T(Xw + b\cdot 1 - y), \quad \partial J/\partial b = (1/n)\, 1^T(Xw + b\cdot 1 - y).$$

Batch update rules. At iteration t the parameters move along the negative gradient with learning rate η:

$$w^{(t+1)} = w^{(t)} - \eta\, \partial J/\partial w, \quad b^{(t+1)} = b^{(t)} - \eta\, \partial J/\partial b.$$

Learning-rate sensitivity and feature standardisation. The Lipschitz constant of the gradient is $(1/n)\cdot\lambda_{max}(X^TX)$. Convergence is guaranteed if and only if $0 < \eta < 2/\lambda_{max}$. On raw features, `test_time` spans [0, 215] while `Jitter(Abs)` spans [1e-6, 1e-4], creating an extreme condition number $\kappa(X^TX) \gg 1e8$; on raw data even $\eta = 0.001$ diverges instantly. Standardising the features to zero mean and unit variance conditions the Hessian and enables stable convergence with $\eta = 0.1$. Table 3.3 reports the learning-rate study on standardised features (3,000 iterations): small rates are stable but slow, $\eta = 0.1$ is both stable and fast, and $\eta \geq 0.5$ diverges within a dozen iterations.

Table 3.3: Gradient-Descent Learning-Rate Study on Standardised Features

Learning rate η	Iterations run	Final training MSE	Outcome
0.001	3,000	97.748	Stable; still converging (slow)
0.01	3,000	94.104	Stable; nearly converged
0.1	3,000	93.958	Stable; converged (selected)
0.5	11	4.39×10^{13}	Diverged
1.5	7	4.16×10^{14}	Diverged

Empirical verification against closed-form OLS. Using $\eta = 0.1$ and 50,000 iterations the scratch GD implementation converged to a final training MSE of **93.9449**, matching the analytical closed-form normal-equation optimum of **93.8967** to within 0.05%. On the held-out test set the two solutions have test RMSE 11.4342 (GD) vs. 11.4414 (closed form), a difference of only 0.0072 UPDRS points; the largest absolute difference between individual predictions is 0.6277 UPDRS points. The remaining gap is consistent with the very flat directions of the loss surface caused by the collinear features, along which gradient descent converges extremely slowly.

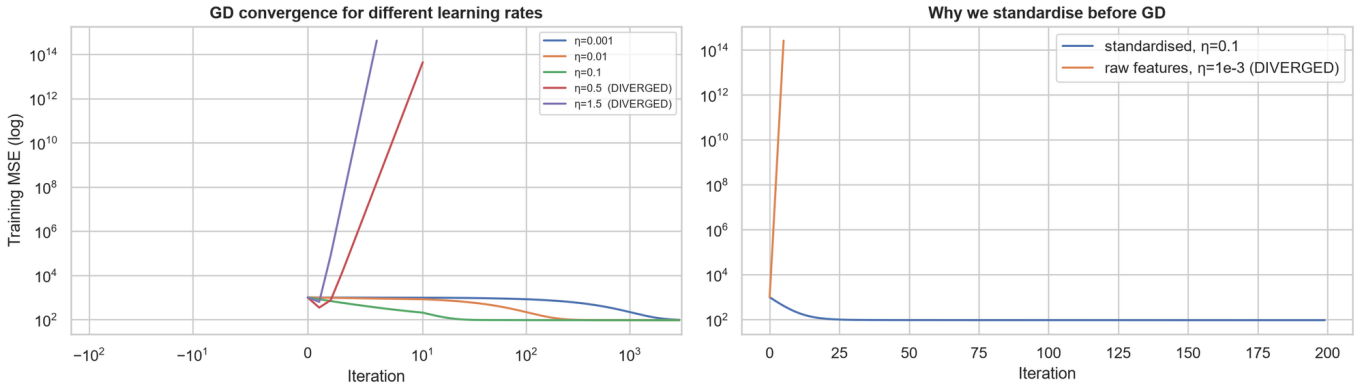


Figure 3.4: Gradient-descent optimisation — (Left) learning-rate trajectories, including the scaling failure on raw data; (Right) loss convergence with and without standardisation.

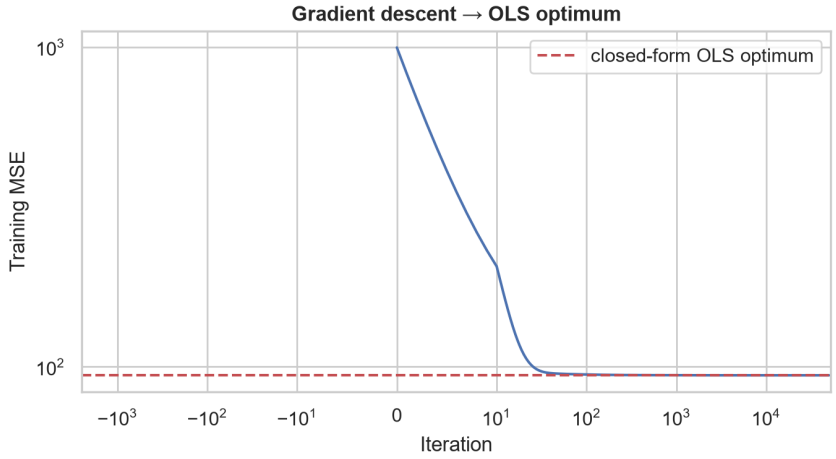


Figure 3.5: Scratch gradient-descent loss curve over 50,000 iterations converging to the closed-form OLS minimum (final MSE 93.94 vs 93.90).

3.4 Polynomial Regression, Bias–Variance Tradeoff, and Overfitting

Polynomial regression expands the linear features into polynomial bases $\varphi(x) = [x_1, x_1^2, x_1x_2, \ldots]$. Because expanding all 19 features to degree p generates C(19+p, p) − 1 features (8,854 terms at degree 4), we conducted a controlled complexity experiment using the **top-5 features most correlated with the target**. The top-5 features were selected by absolute Pearson correlation with `total_UPDRS` computed on the **training patients only** (33 patients), so no test-patient information influenced feature selection (Table 3.4).

Table 3.4: Correlation-Based Feature Ranking (training patients only; top 5 are used in the polynomial study)

Rank	Feature	Pearson r with `total_UPDRS` (train patients)
1	`age`	**+0.402**
2	`PPE`	**+0.183**
3	`HNR`	**−0.129**
4	`Shimmer:APQ11`	**+0.110**
5	`test_time`	**+0.105**
6	`RPDE`	+0.089
7	`Shimmer(dB)`	+0.087
8	`Jitter(%)`	+0.085

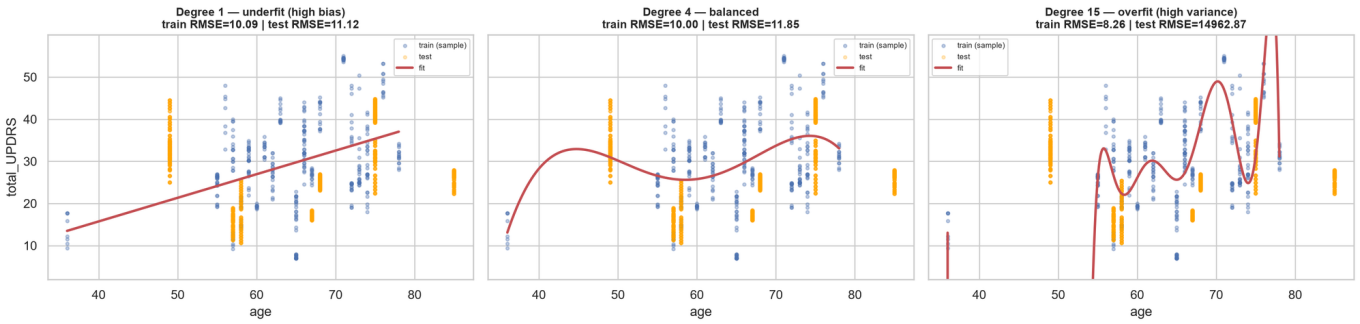


Figure 3.6: Single-feature polynomial fit (`age`) — degree 1 underfits (train RMSE 10.09, test RMSE 11.12), degree 4 balances (10.00 / 11.86), and degree 15 oscillates wildly (8.26 / 14,962.87).

Table 3.5: Multi-Feature Polynomial Degree Study (Top-5 Features, Degrees 1 to 8)

Degree	Terms	Train RMSE	5-Fold CV RMSE	CV RMSE SD	Test RMSE	Train R^2	Test R^2
1 (Linear)	5	9.9026	**10.2612**	1.7075	11.4236	0.1927	−0.5749
2	20	9.5901	10.8111	1.6374	12.7086	0.2429	−0.9491
3	55	9.2979	15.3128	5.2949	13.7612	0.2883	−1.2854
4	125	8.7259	68.5458	103.7892	16.1720	0.3732	−2.1563
5	251	8.4527	270.6261	460.1585	80.6724	0.4118	−77.5410
6	461	8.1319	1,473.5014	2,415.8266	1,470.3896	0.4556	−26,091.27
7	791	7.6945	5,801.3840	6,234.2047	14,449.12	0.5126	−2.52e+06
8	1286	6.6361	141,427.03	192,637.51	340,234.48	0.6375	−1.40e+09

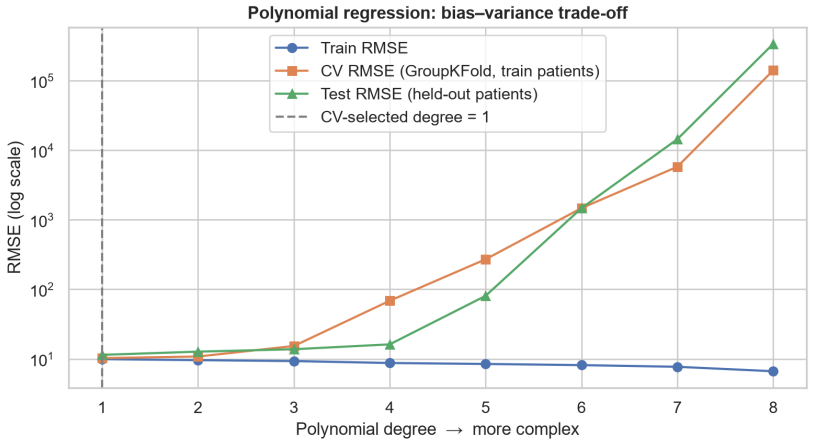


Figure 3.7: Polynomial degree vs. RMSE (log scale) showing monotonic training-error descent contrasted against catastrophic validation/test divergence.

Analysis of bias–variance dynamics. As the polynomial degree increases from 1 to 8, the training RMSE steadily drops from 9.90 to 6.64 (R^2 rises from 0.19 to 0.64) because the 1,286 basis terms fit training idiosyncrasies. Concurrently, the test RMSE explodes to 340,234.5 (R^2 = −1.40e9). Cross-validation over patient folds firmly selects **degree 1** as optimal (CV RMSE = 10.2612). Higher-degree unregularised polynomials suffer explosive variance and severe out-of-distribution extrapolation outside the training feature support.

Underfitting (high bias). A linear model on the top-5 features (degree 1) reaches only a train R^2 of 0.1927 (train RMSE 9.90 against 11.02 for the mean predictor). The single-feature degree-1 fit on `age` (Figure 3.6, left) is likewise too rigid to follow the data. In §3.5 the strongly regularised Poly Deg 2 + Ridge ($\alpha = 10^5$) is an extreme case, with a train R^2 of 0.0418, an almost flat predictor. Both have high bias and low variance. Degrees 4 to 8 show the opposite regime (overfitting). Table 3.5 and Figure 3.7 together span the full bias–variance range, and Table 3.6 collects the underfitting, well-fitted and overfitting cases of this assignment in one place.

Table 3.6: Underfitting and Overfitting Cases in the Regression Study

Regime	Model	Train R^2	Test RMSE	Diagnosis
Underfitting	Linear, top-5 features (degree 1)	0.1927	11.4236	High bias; cannot capture non-linear structure
Underfitting	Poly Deg 2 + Ridge ($\alpha = 10^5$)	0.0418	9.3416	Near-constant predictor; shrinks to the mean
Best fit (by CV)	Ridge (CV α = 2682.7)	0.1818	9.9978	Balanced bias and variance; best mean CV RMSE
Overfitting	Polynomial degree 8, top-5 features	0.6375	340,234.48	Training error falls, test error explodes
Overfitting	Poly Deg 2, all features, OLS (209 terms)	0.4490	35.1111	High variance from collinear polynomial terms
Overfitting (leakage)	Random Forest, random-row split	0.9774 (test R^2)	1.5812	Memorises patient identity; collapses to R^2 = −1.1656 on unseen patients

3.5 Regularisation: Ridge, Lasso, and Coefficient Dynamics

To tame multicollinearity and polynomial explosion we evaluate L2 (Ridge) and L1 (Lasso) regularised regression, using the scikit-learn formulation:

$$\text{Ridge: } \min_{w,b} \|y - Xw - b\|^2 + \alpha\|w\|^2_2, \quad \text{Lasso: } \min_{w,b} (1/2n)\|y - Xw - b\|^2 + \alpha\|w\|_1.$$

Ridge shrinks all coefficients smoothly and distributes weight across correlated features; Lasso additionally drives coefficients exactly to zero and so performs feature selection. The parameter α was tuned by 5-fold `GroupKFold` (grouping by patient) across 50 logarithmically spaced values:

- **Ridge:** optimal α = **2682.70** (CV RMSE 10.681); the 1-standard-error rule gives α = 10,000.
- **Lasso:** optimal α = **1.2068** (CV RMSE 10.354); the 1-standard-error rule gives α = 10 (matching Figure 3.8).

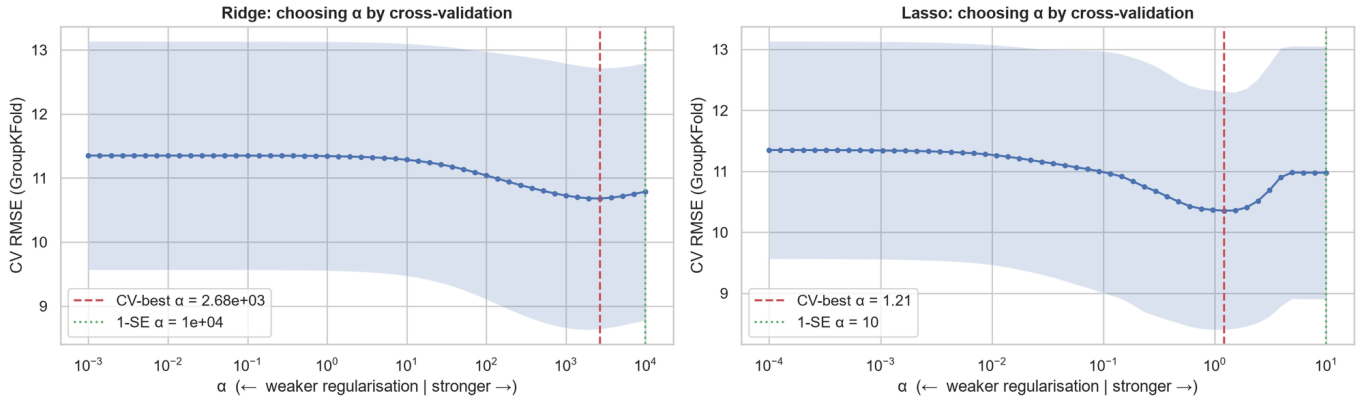


Figure 3.8: Regularisation parameter selection via GroupKFold CV for Ridge (left) and Lasso (right), depicting the optimal and 1-SE α.

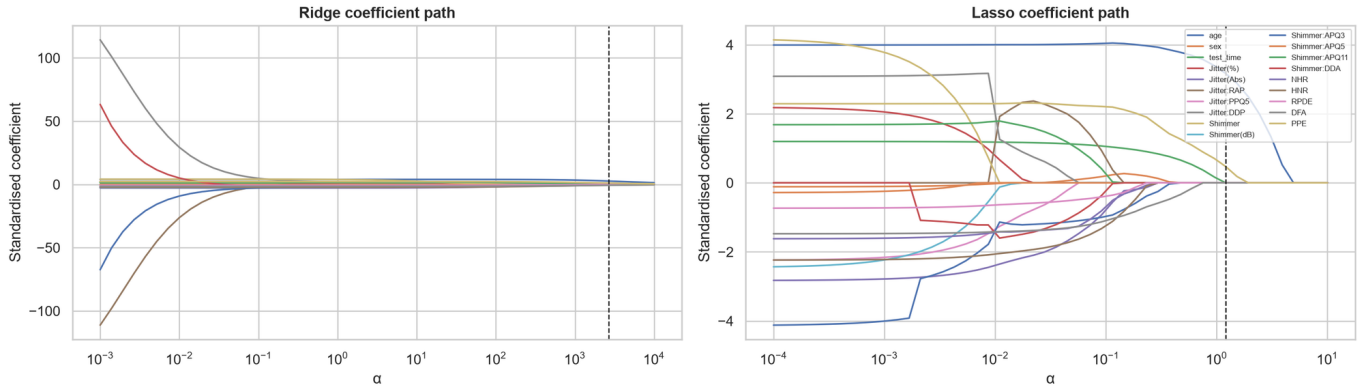


Figure 3.9: Standardised coefficient traces across α — Ridge shrinks smoothly; Lasso drives coefficients to exactly zero.

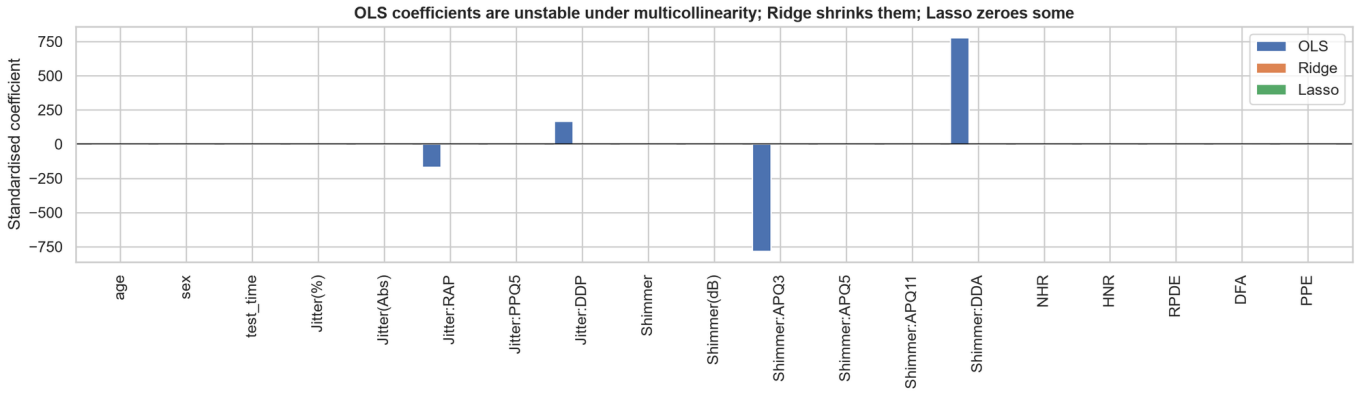


Figure 3.10: Comparison of standardised feature weights across OLS, Ridge, and Lasso under extreme multicollinearity.

Lasso sparsity and feature elimination. Under Lasso with CV-selected α = 1.2068, **17 of the 19 features are driven to exactly zero**. The only features retained are `age` (w = +3.1689) and `PPE` (w = +0.4721). All 15 collinear jitter/shimmer/noise measures are completely zeroed out. Notably, `age`, which is constant per patient, carries the largest surviving coefficient, again reflecting that the signal in this data set is primarily between patients rather than within a patient.

Curing polynomial overfitting with regularisation. Expanding all 19 features to degree 2 yields 209 features. Unregularised OLS on this polynomial matrix suffers severe test degradation (test RMSE 35.11, R^2 = −13.88). For Ridge, the CV optimum sits at the edge of the α grid (α = 100,000) and the model is close to a constant predictor (see §3.6; test RMSE = 9.3416). Applying Lasso (α = 2.2603) retains only 1 of the 209 polynomial terms and achieves a test RMSE of 9.7788. Regularisation therefore removes the variance explosion completely, but it does so mainly by shrinking the model towards the mean predictor.

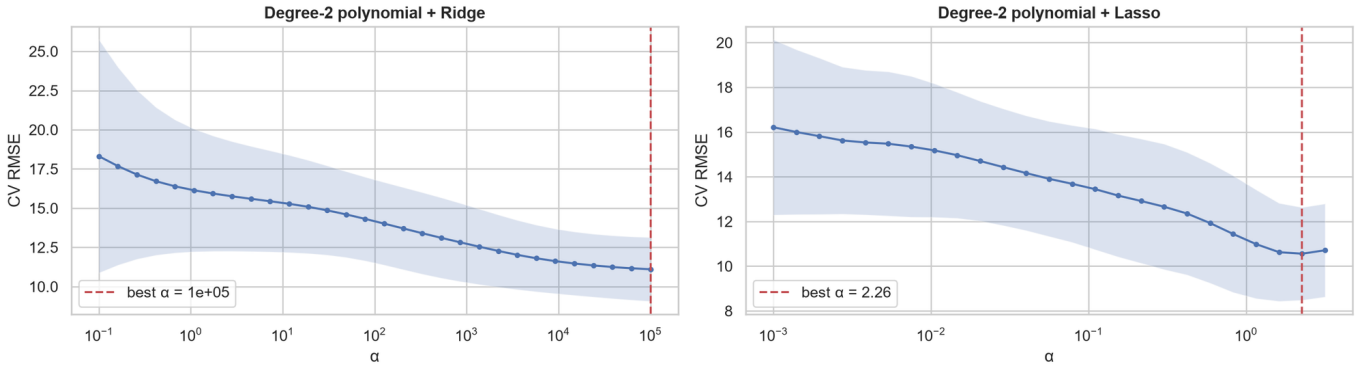


Figure 3.11: Regularisation parameter selection for the degree-2 polynomial pipeline (209 features) via GroupKFold CV.

3.6 Benchmark Synthesis and the Honest Reality of UPDRS Prediction

Table 3.7 summarises the evaluation of all regression models on the held-out test patients (N = 1,235). MSE, RMSE and R^2 are reported for each model, together with the Pearson correlation between predictions and truth.

Table 3.7: Complete Regression Model Comparison on Held-Out Patients (N = 1,235)

Model specification	Test MSE	Test RMSE	Test R^2	Pearson R	Train RMSE	Train R^2
Baseline (train-mean predictor)	89.6240	9.4670	−0.0816	NaN	11.0213	0.0000
Linear regression (GD from scratch)	130.7405	11.4342	−0.5778	0.1195	9.6925	0.2266
Linear regression (closed-form OLS)	130.9058	11.4414	−0.5798	0.1187	9.6900	0.2270
Polynomial (Deg 1, top-5 feats)	130.4986	11.4236	−0.5749	0.1044	9.9026	0.1927
Ridge (CV α = 2682.7)	99.9557	9.9978	−0.2063	0.1359	9.9692	0.1818
Lasso (CV α = 1.2068)	106.0178	10.2965	−0.2795	0.1005	10.1045	0.1595
Poly Deg 2 (all feats) OLS	1232.7901	35.1111	−13.8777	0.0361	8.1813	0.4490
Poly Deg 2 + Ridge (α = 100000)	**87.2657**	**9.3416**	**−0.0532**	**0.1560**	10.7887	0.0418
Poly Deg 2 + Lasso (α = 2.2603)	95.6243	9.7788	−0.1540	0.0876	10.3437	0.1192

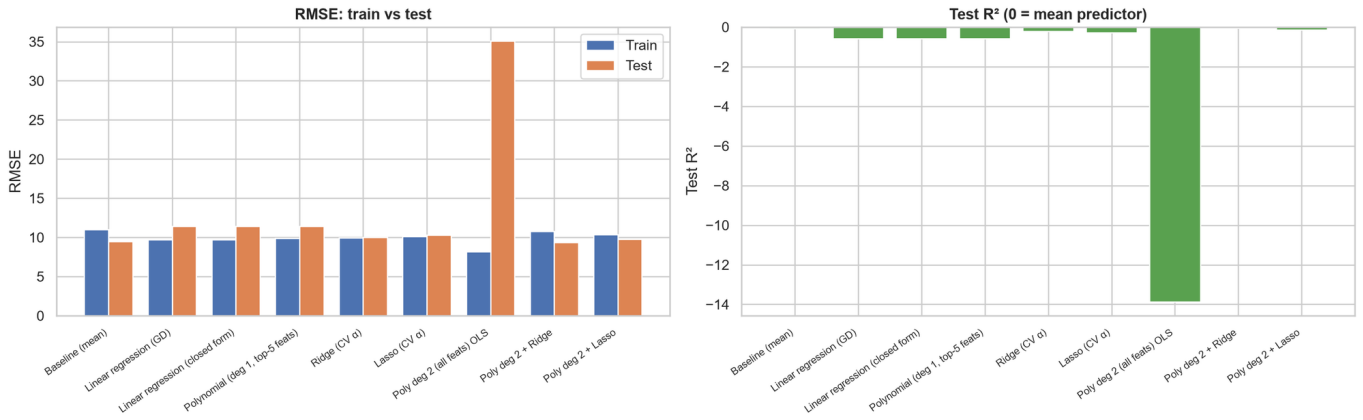


Figure 3.12: Model comparison on held-out patients — (Left) RMSE train vs. test; (Right) test R^2, showing persistent negative values relative to the baseline.

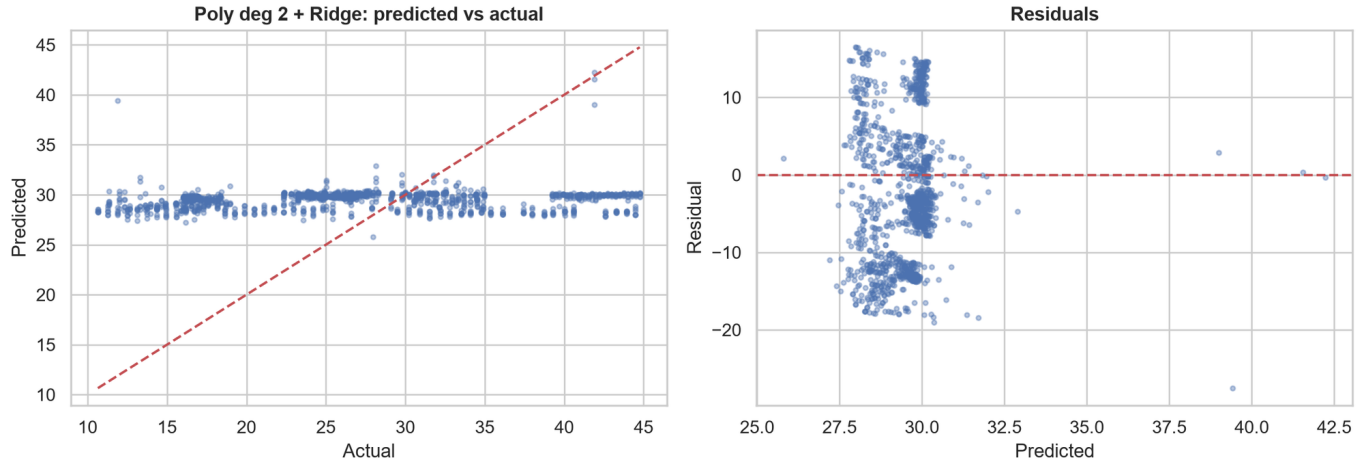


Figure 3.13: Diagnostic plots for the regularised polynomial (Poly Deg 2 + Ridge, α = 100,000) — (Left) predicted vs. actual; (Right) residuals on held-out patients.

Table 3.8: 5-Fold GroupKFold Cross-Validation Robustness Across All 42 Patients

Model	Mean CV RMSE	CV RMSE SD	Mean CV R^2	CV R^2 SD
Baseline (train-mean predictor)	10.9202	1.1160	−0.2144	0.3042
Linear regression (OLS)	11.2531	1.4587	−0.3079	0.3923
Ridge (CV α)	**10.6818**	1.3702	**−0.1589**	0.2797
Lasso (CV α)	10.7422	1.1587	−0.1722	0.2737
Poly Deg 2 + Ridge	11.1398	1.3389	−0.2560	0.2870
Poly Deg 2 + Lasso	11.1793	1.0994	−0.2652	0.2562

Interpretation of negative R² scores

In regression, $R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y}_{test})^2$. A negative R^2 indicates that the model's predictions on unseen test subjects have a larger mean squared error than simply predicting the constant test-set mean.

1. **Baseline shift.** The naive DummyRegressor predicting the training mean achieves test RMSE = 9.4670 (R^2 = −0.0816). The negative R^2 arises directly from the inter-subject baseline shift: the training-cohort target mean is 29.5656 (std 11.0213), whereas the held-out test-cohort mean is 26.9651 (std 9.1028), a gap of 2.6005 UPDRS points that adds a constant bias offset ($2.6005^2 \approx 6.76$) to the test MSE.

2. **Absence of reliable predictive superiority.** Although Poly Deg 2 + Ridge achieves a test RMSE of 9.3416 on this single split, its regulariser is α = 100,000 (the maximum of the search grid), producing an almost flat predictor (train R^2 = 0.0418) that heavily shrinks predictions toward the mean. In the 5-fold GroupKFold cross-validation across all 42 patients (Table 3.8), the tuned Ridge (10.68 ± 1.37) and Lasso (10.74 ± 1.16) are marginally better than the mean baseline (10.92 ± 1.12), but the gaps (0.24 and 0.18 RMSE) are well within one fold standard deviation. Since α was tuned on the 33 training patients, which overlap with the validation folds of this 42-patient CV, these estimates are mildly optimistic. We therefore conclude that no model shows a reliable improvement over the naive mean predictor on unseen patients; the polynomial and unregularised models are clearly worse.

3. **Scientific conclusion.** Voice features and demographics alone did not reliably predict total UPDRS across unseen patients in this experiment. Clinically, this is consistent with the strong patient-specific baseline: a useful monitoring model would likely need a per-patient calibration step, for example an initial clinic visit that anchors the patient's baseline score.

3.7 Summary of Q2 Findings

Assignment item	Finding
(a) Gradient descent	On standardised features with $\eta = 0.1$ and 50,000 iterations the scratch implementation reaches training MSE 93.9449 vs. 93.8967 (closed form); raw features diverge.
(b) Polynomial regression	Patient-wise CV selects degree 1; degree 8 reaches train R^2 0.64 but test RMSE 340,234 (Table 3.5).
(c) Ridge and Lasso	α chosen by GroupKFold: Ridge 2682.7, Lasso 1.2068; Lasso retains only age and PPE (17 of 19 coefficients exactly zero).
(i) MSE, RMSE, R^2	Table 3.7. Best single-split test RMSE 9.3416 (Poly Deg 2 + Ridge, near-constant predictor) vs. 9.4670 for the mean baseline; all R^2 are negative or ≈ 0.
(ii) Overfitting / underfitting	Table 3.6, Figures 3.6 and 3.7, and the leakage experiment of Table 3.1.
(iii) CV for regularisation	Five-fold GroupKFold over 50 log-spaced α values (Figures 3.8 and 3.11); 1-SE rule values also reported.

4. Q3 — Unsupervised K-Means Clustering and PCA from Scratch

4.1 Dataset Cleaning, Feature Engineering, and Continuous Scaling

The Netflix Titles dataset comprises 8,807 media entries described across 12 raw attributes (`show_id`, `type`, `title`, `director`, `cast`, `country`, `date_added`, `release_year`, `rating`, `duration`, `listed_in`, `description`). Because clustering and PCA operate on real coordinate spaces R^d, a comprehensive feature transformation was implemented. Missing values occur in six raw columns (Table 4.1); `director`, `cast` and `country` are the sparsest, which is why they are represented through indicator flags (including an explicit `Unknown` country flag) rather than being dropped.

Table 4.1: Missing Values in the Raw Netflix Table and Their Treatment

Raw column	Missing entries	Share of 8,807	Treatment
`director`	2,634	29.9%	Not used as a feature (high-cardinality text)
`cast`	825	9.4%	Not used as a feature (high-cardinality text)
`country`	831	9.4%	Multi-hot flags; `country_Unknown` indicator
`date_added`	10	0.1%	Parsed to `added_year`
`rating`	4	< 0.1%	3 misplaced durations repaired; mapped to 5 rating groups
`duration`	3	< 0.1%	Restored from the misplaced `rating` entries; parsed to minutes / seasons

- **Data repair.** In three anomalous records the duration (e.g. '74 min') was misplaced into the `rating` column while `duration` was null; these were parsed and restored.
- **Format and duration.** The binary indicator `is_movie` (1 = Movie, 0 = TV Show) is complemented by the continuous metrics `movie_minutes` and `tv_seasons`, extracted with 1st–99th percentile Winsorisation within type to neutralise outlier leverage.
- **Rating taxonomy.** The 14 raw MPAA/TV parental ratings were collapsed into 5 coherent categories — `Kids` (TV-Y, G), `Family` (PG, TV-PG), `Teen` (PG-13, TV-14), `Adult` (R, TV-MA, NC-17) and `Unrated` — encoded as 5 binary flags.
- **Multi-hot genre and country representations.** Titles exhibit 1–3 genres and multiple production countries. We retained the 40 of 42 genres that appear in at least 50 titles (e.g. `International Movies`, `Dramas`, `Comedies`) and the 15 top production countries plus an `Unknown` indicator, represented as multi-hot binary vectors.

Table 4.2: Composition of the 66-Dimensional Feature Matrix

Feature group	Number of features	Type	Scaling
Format indicator (`is_movie`)	1	Binary	None
Continuous (`release_year`, `added_year`, `movie_minutes`, `tv_seasons`)	4	Continuous	z-score (zero mean, unit variance)
Rating groups	5	Binary	None
Genres (≥ 50 titles)	40	Multi-hot binary	None
Production countries (top 15 + Unknown)	16	Multi-hot binary	None
Total	**66**	4 continuous + 62 binary	

The scaling principle. Distance-based algorithms (K-Means) and variance-maximising projections (PCA) are highly sensitive to coordinate scaling. Crucially, we standardise (z-score) **only the 4 continuous numeric features**. The 62 binary indicators (0/1) are left unstandardised: if a rare binary indicator (e.g. a genre occurring in 1% of titles) were z-scored, its standardised coordinates would stretch to about ±10 and allow negligible categories to dominate Euclidean distances.

4.2 Vectorised K-Means from Scratch and scikit-learn Validation

Lloyd's algorithm. K-Means partitions n points into K disjoint clusters $C_1, \ldots, C_K$ to minimise the Within-Cluster Sum of Squares (WCSS, or inertia):

$$WCSS = \Sigma_{k=1..K} \Sigma_{x_i \in C_k} \|x_i - \mu_k\|^2, \quad \mu_k = (1/|C_k|) \Sigma_{x_i \in C_k} x_i.$$

The implementation includes four design features:

- **Fully vectorised Euclidean distances.** Instead of looping over points and centroids we use the identity $\|x - c\|^2 = \|x\|^2 - 2\, x \cdot c^T + \|c\|^2$, computed through a single matrix multiplication `(n, d) @ (d, K)` in NumPy.
- **K-Means++ seeding.** The first centroid is sampled uniformly at random; subsequent centroids are sampled with probability proportional to the squared Euclidean distance to the nearest existing centroid, $p(x) \propto D(x)^2$, which disperses the initial centres.

- **Empty-cluster recovery.** If a cluster becomes vacant, its centroid is re-seeded at the data point with the maximum distance from its current centroid, preventing dimensional collapse.
- **Multi-start convergence.** n_init = 10 independent runs are executed and the run with the minimum WCSS is retained.

Each iteration can only decrease the WCSS (Appendix C.3), so the algorithm converges to a local minimum; Figure 4.1 (right) shows the monotone WCSS decrease for K = 4 over 16 iterations. Because the local minimum depends on the initialisation, the left panel of Figure 4.1 compares single-run results from 40 random and 40 k-means++ starts.

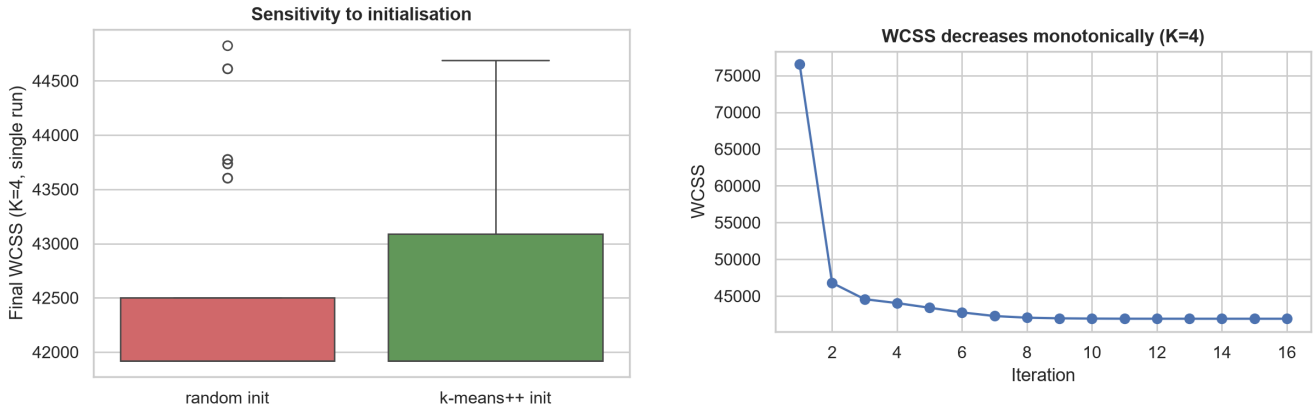


Figure 4.1: (Left) Sensitivity of single-run WCSS at K = 4 across 40 random vs. k-means++ starts; (Right) monotonic convergence of WCSS over iterations.

Table 4.3: Sensitivity of Single-Run K-Means to Initialisation (40 starts each)

Initialisation	Mean final WCSS (K = 4)	Std. dev.	Best WCSS
Random	42,437	901	41,918
k-means++	42,456	794	41,918

On this data set k-means++ reduces the spread of outcomes (standard deviation 794 vs. 901) but does not improve the mean, and both strategies reach the same best value of 41,918 once multiple starts are taken. This is why the multi-start safeguard (n_init = 10) is used for all reported results.

Table 4.4: Validation of the Scratch K-Means Implementation vs. scikit-learn

K	WCSS (scratch)	WCSS (scikit-learn)	Relative WCSS difference	Adjusted Rand Index (ARI)	Iterations (scratch)
2	50994.7659	50994.7659	0.0000 %	1.000000	7
3	45671.9165	45671.9992	−0.0002 %	0.999830	15
4	41917.8861	41917.8861	−0.0000 %	1.000000	16
5	38700.2009	38700.2593	−0.0002 %	0.998541	14
6	36900.6579	36900.6496	+0.0000 %	0.999670	23

Validation against scikit-learn

Table 4.4 provides strong numerical evidence for the correctness of the scratch implementation. Across all required K in [2, 6], the relative WCSS deviation from `sklearn.cluster.KMeans` is bounded within ±0.0002%, and the Adjusted Rand Index reaches near-unity (1.0000 at K = 2 and K = 4; above 0.9985 for all K), confirming near-identical partitions. The residual differences arise from different random initialisations converging to marginally different local minima.

4.3 WCSS Analysis, Elbow Curve, and Cluster Profiling

Table 4.5 details the WCSS, the marginal percentage reduction, and the mean silhouette score for K = 2…6, as required by the assignment.

Table 4.5: Required WCSS and Silhouette Scores for K = 2 to 6 (Netflix Dataset, N = 8,807)

Cluster count (K)	WCSS (inertia)	Marginal drop vs. K−1 (%)	Mean silhouette	Interpretation
K = 2	50,994.77	23.61 %	0.2572	Highest silhouette; partitions Movie vs. TV Show.
K = 3	45,671.92	10.44 %	0.2538	Primary elbow inflection on the extended curve; silhouette remains high.
K = 4	**41,917.89**	**8.22 %**	0.1485	Secondary knee in the required grid; lower silhouette than K = 2 and K = 3.
K = 5	38,700.20	7.68 %	0.1596	Marginal gain drops below 8%; slight silhouette uptick.
K = 6	36,900.66	4.65 %	0.1431	Marginal gain drops below 5%; lowest silhouette in the grid.

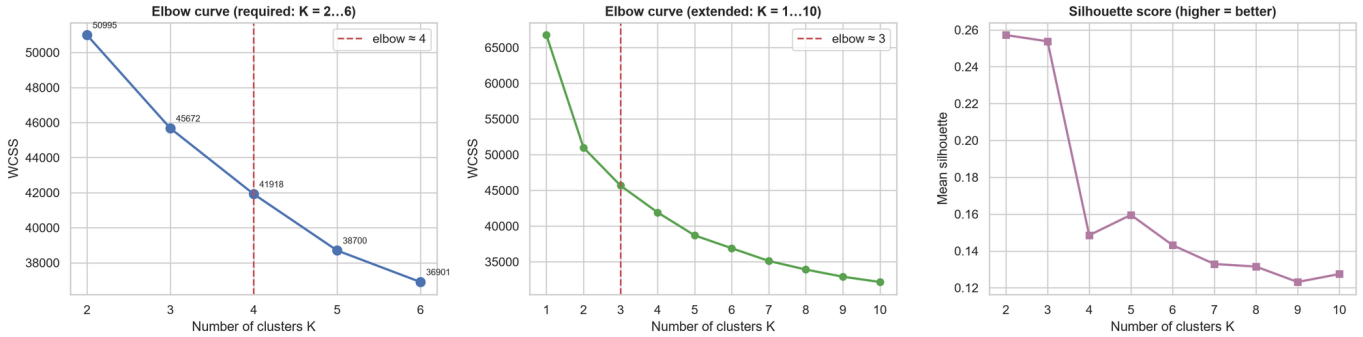


Figure 4.2: Elbow curves and silhouette analysis — (Left) required K = 2…6; (Middle) extended K = 1…10 showing a smooth decline; (Right) silhouette trajectory.

The nature of the "soft" elbow. In real-world catalog data comprising 62 sparse binary flags, clusters do not form spherical, well-separated Gaussian clouds. Consequently the elbow curve does not exhibit a sharp right-angled bend. The largest drop occurs at K = 2 (23.61%), which merely separates Movies from TV Shows; this is trivial and uninformative for content recommendation. The geometric distance to the chord connecting K = 2 and K = 6 identifies **K = 4 as the candidate elbow within the required range**, beyond which the marginal gain drops below 7.7%. Two scientific caveats must be disclosed: (1) the silhouette score at K = 4 (0.1485) is substantially lower than at K = 2 (0.2572) and K = 3 (0.2538), reflecting cluster overlap in sparse multi-hot genre space; (2) on the extended evaluation grid (K = 1…10, Figure 4.2 middle) the primary elbow occurs at K = 3 (a drop of 10.44% following the 23.61% drop from K = 1 to 2), making K = 4 a secondary knee choice.

Empirical cluster profiling (K = 4)

Examining the empirical cluster centroids (`q3_cluster_profiles.csv`), we assign descriptive names strictly supported by the data distributions (Table 4.6). The names are our interpretation.

Table 4.6: Empirical Cluster Profiles for K = 4 (from `q3_cluster_profiles.csv`)

Cluster & assigned name	Size (%)	% Movies	Median year	Median runtime (min)	Top rating	Distinguishing genre & lift	Top origins
Cluster 0: Global mainstream dramas & romances	3,286 (37.3%)	100.0%	2018	99	Adult	Int'l Movies (45%), Dramas (41%); Romantic Movies ×1.7, Children & Family ×1.7	US 43%, India 13%
Cluster 1: Classic & heritage cinema	572 (6.5%)	96.9%	1992	109	Teen	Classic Movies (×15.3 lift), Cult Movies (×10.4 lift)	US 60%, India 18%
Cluster 2: International & domestic TV series	2,655 (30.1%)	0.0%	2018	—	Adult	Int'l TV Shows (51%), TV Dramas (29%); Romantic TV ×3.3, Korean TV ×3.3	US 35%, Unknown 15%
Cluster 3: Documentaries, stand-up & indie features	2,294 (26.0%)	99.9%	2016	95	Adult	Int'l Movies (47%), Dramas (38%); Stand-Up Comedy ×2.2, Documentaries ×2.0	US 43%, India 19%

The cluster names are our interpretation, and clusters 0 and 3 have very similar top genres and countries. Representative titles reinforce the labels: cluster 0 includes 'Cop Car' and 'Sorry To Disturb'; cluster 1 includes 'The Lord of the Rings: The Return of the King' and 'The Long Riders'; cluster 2 includes 'Friday Night Lights' and 'Julie and the Phantoms'; cluster 3 includes 'The Meaning of Monty Python' and 'Sarah Silverman A Speck of Dust'.

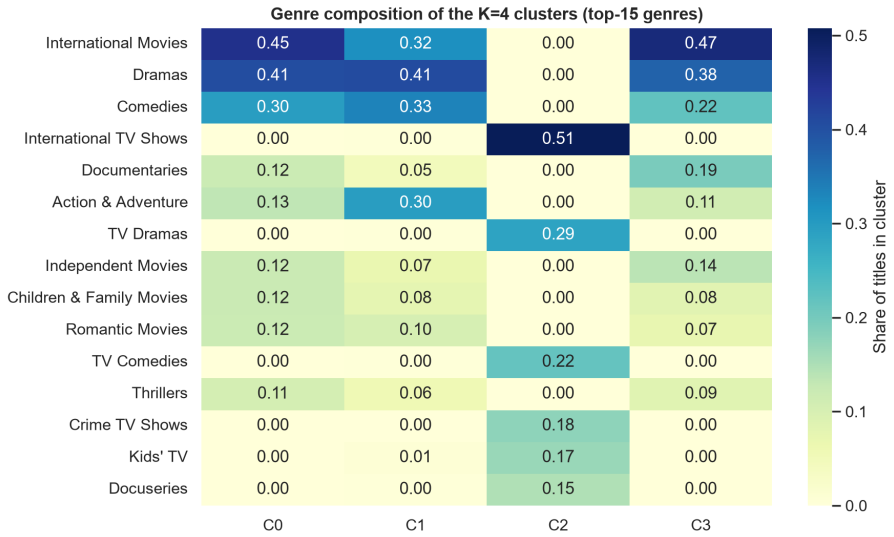


Figure 4.3: Genre-composition heatmap across the K = 4 clusters, illustrating the clear separation of TV titles and classic movies; the two large movie clusters differ mostly in genre mix.

4.4 Principal Component Analysis (PCA) from Scratch

Algorithm via covariance decomposition.

1. *Centering:* compute the empirical mean $\mu = (1/n)\,\Sigma\, x_i$ and form the centred matrix $X_c = X - 1\mu^T$.
2. *Covariance matrix:* construct the symmetric positive semi-definite matrix $\Sigma = (1/(n-1))\, X_c^T X_c \in R^{p\times p}$ (here $p = 66$).
3. *Eigendecomposition:* solve $\Sigma v_j = \lambda_j v_j$ with `np.linalg.eigh`, which guarantees real eigenvalues and orthonormal eigenvectors $V = [v_1, \ldots, v_p]$ with $V^T V = I$.
4. *Ordering:* sort $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_p \geq 0$ and arrange the principal axes accordingly.
5. *Explained variance ratio:* $r_j = \lambda_j / \Sigma_m \lambda_m$; the cumulative variance is $C_k = \Sigma_{j\leq k}\, r_j$.
6. *Orthogonal projection:* $Z_k = X_c V_k$ with $V_k = [v_1, \ldots, v_k]$; the rank-k reconstruction is $\hat{X} = Z_k V_k^T + 1\mu^T$.

Mathematical verification checks

- Eigen-equation residual $\max |\Sigma v - \lambda v|$ is of order 10^{-15} (below 10^{-14}), i.e. machine precision.
- Orthonormality deviation $\max |V^T V - I|$ is likewise of order 10^{-15}, confirming an orthonormal basis.
- Trace equality: $\Sigma_{j=1..66}\, \lambda_j = \text{trace}(\Sigma) \approx 7.5805$, so the total variance is preserved exactly.
- The explained variance ratios match `sklearn.decomposition.PCA` (`np.allclose(atol=1e-8)` is True) and the first 10 principal axes agree with scikit-learn's up to sign ($|\cos| = 1$).
- The variance of the projected coordinates equals the corresponding eigenvalues.

Table 4.7: Leading Eigenvalues, Explained Variance Ratios and Cumulative Variance (66-D feature matrix)

Component	Eigenvalue λ	Explained variance ratio	Cumulative
PC1	2.1223	0.2800	0.2800
PC2	1.0890	0.1437	0.4236
PC3	0.8895	0.1173	0.5410
PC4	0.4459	0.0588	0.5998
PC5	0.3844	0.0507	0.6505
PC6	0.3341	0.0441	0.6946
PC7	0.2102	0.0277	0.7223
PC8	0.1675	0.0221	0.7444
PC9	0.1500	0.0198	0.7642
PC10	0.1360	0.0179	0.7822

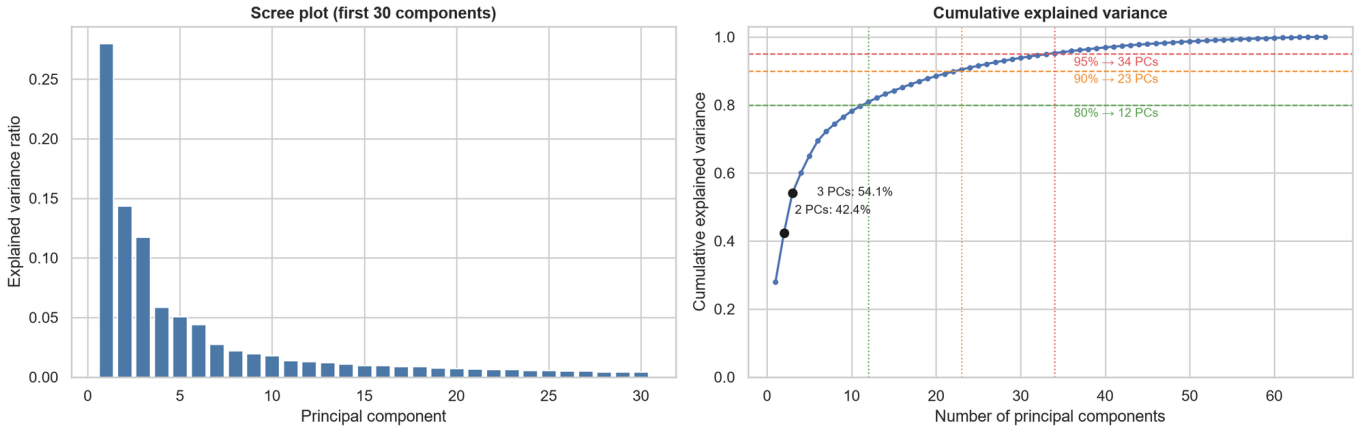


Figure 4.4: (Left) Scree plot for the first 30 components; (Right) cumulative explained variance curve marking the 80%, 90% and 95% thresholds.

Table 4.8: Principal-Component Variance Decomposition and Dimensionality-Reduction Summary

Component	Eigenvalue (λ)	Explained variance (%)	Cumulative variance (%)	Variance retained	Reconstruction MSE
PC1	2.1223	27.997 %	27.997 %	—	—
PC2	1.0890	14.366 %	42.363 %	**42.36 % (2-PC space)**	**0.0662**
PC3	0.8895	11.734 %	54.098 %	**54.10 % (3-PC space)**	**0.0527**
PC4	0.4459	5.883 %	59.981 %	—	—
PC5	0.3844	5.071 %	65.052 %	—	—
PC12 (80% threshold)	0.0989	1.305 %	80.890 %	80.89 % (12 PCs)	—
PC23 (90% threshold)	0.0473	0.624 %	90.412 %	90.41 % (23 PCs)	—
PC34 (95% threshold)	0.0246	0.325 %	95.233 %	95.23 % (34 PCs)	—

Variance retention and dimensionality reduction

- **2-component reduction** retains 42.36% of the total variance, with a reconstruction error (MSE) of 0.0662. The reduced space has 64 fewer dimensions.
- **3-component reduction** retains 54.10% of the total variance, with a reconstruction MSE of 0.0527 (63 fewer dimensions).
- **Consistency check.** The per-entry reconstruction MSE of a rank-k projection equals the discarded eigenvalue mass divided by p: (7.5805 − 3.2113)/66 = 0.0662 for k = 2 and (7.5805 − 4.1008)/66 = 0.0527 for k = 3, matching the reported values exactly.
- **Higher thresholds.** Capturing 80% of the total variance requires 12 components, 90% requires 23 and 95% requires 34. In a 66-dimensional space dominated by sparse multi-hot genre and country flags, the variance is naturally distributed over dozens of orthogonal axes; capturing more than 54% in just three dimensions still represents significant compression.

4.5 2-D / 3-D Projections, Loadings Interpretation, and K-Means in PCA Space

Figure 4.5 presents the projection of all 8,807 titles onto PC1 and PC2, stratified by content type, K-Means cluster and rating group. Figure 4.6 shows the first three components for a 3,000-title sample.

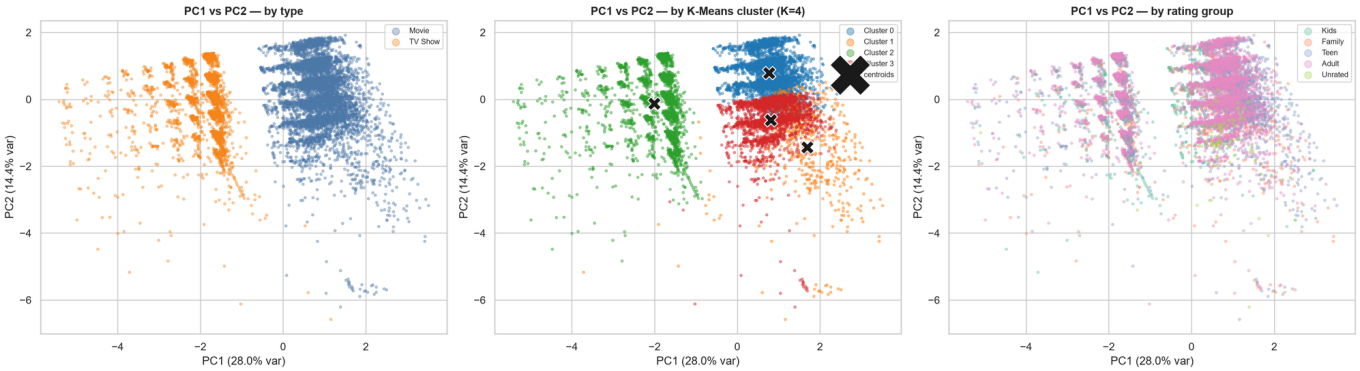


Figure 4.5: 2-D principal-component projection (PC1 vs. PC2) — (Left) partition by type; (Middle) partition by K-Means cluster with centroids; (Right) partition by rating group.

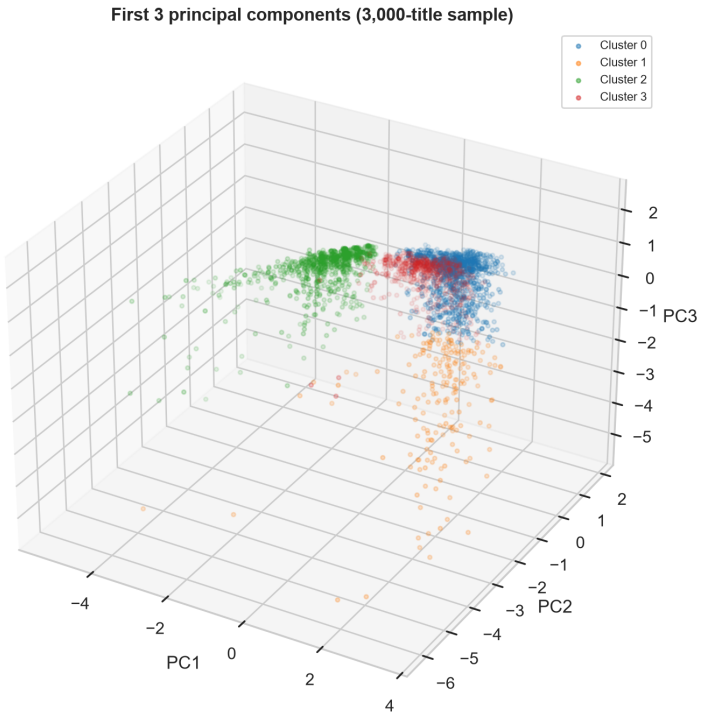


Figure 4.6: 3-D principal-component scatter plot of 3,000 catalog samples coloured by K-Means cluster.

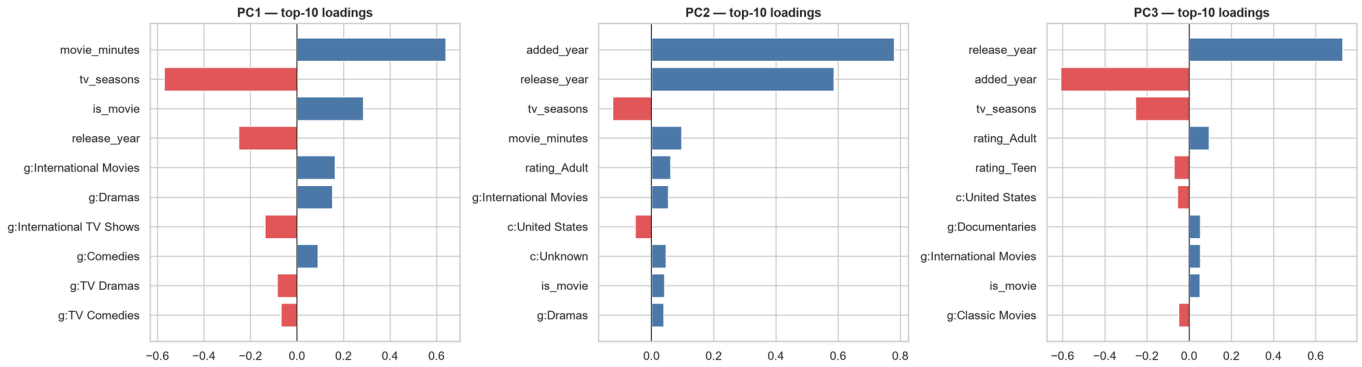


Figure 4.7: Top-10 component loadings (eigenvector elements) for PC1, PC2 and PC3.

Table 4.9: Leading Loadings of the First Three Principal Components (from `q3_pca_loadings_first3.csv`)

Component	Strongest positive loadings	Strongest negative loadings	Interpretation
PC1 (28.0%)	`movie_minutes` +0.6408; `is_movie` +0.2862; `genre_International Movies` +0.1645; `genre_Dramas` +0.1521	`tv_seasons` −0.5714; `release_year` −0.2506; `genre_International TV Shows` −0.1372; `genre_TV Dramas` −0.0859	Media format axis: movies vs. television
PC2 (14.4%)	`added_year` +0.7810; `release_year` +0.5872	`tv_seasons` −0.1255; `country_United States` −0.0523	Catalog recency: modern titles added recently
PC3 (11.7%)	`release_year` +0.7280	`added_year` −0.6082; `tv_seasons` −0.2553	Gap between production year and ingestion into the catalog

Semantic interpretation of the principal axes

- **PC1 (28.0% of variance) — the media format axis.** PC1 separates movies from television. The strongest positive loadings are `movie_minutes` (+0.6408) and `is_movie` (+0.2862), alongside the movie genres; the strongest negative loadings are `tv_seasons` (−0.5714), `release_year` (−0.2506) and TV genres.
- **PC2 (14.4%) — the catalog recency axis.** PC2 captures temporal freshness: both `added_year` (+0.7810) and `release_year` (+0.5872) load strongly positive, opposed by `tv_seasons` (−0.1255) and `country_United States` (−0.0523). High PC2 values correspond to modern titles added recently to the catalog.
- **PC3 (11.7%) — the release-year versus added-year axis.** Rather than geography, PC3 reflects the time gap between production and ingestion. `release_year` loads strongly positive (+0.7280) while `added_year` loads strongly negative (−0.6082), followed by `tv_seasons` (−0.2553). The axis separates recently released productions from older classic or archival acquisitions added late to the platform.

Clustering in reduced PCA space. To test geometric preservation we evaluated K-Means (K = 4) in the 3-D and 2-D PCA spaces (`results/q3_kmeans_full_vs_pca.csv`, Table 4.10).

Table 4.10: K-Means Clustering Performance across Full and Reduced Latent Spaces

Feature space	Dimensions	Mean silhouette	ARI vs. full 66-D clustering	Final WCSS
Full coordinate space	66	0.1485	1.0000 (reference)	41,917.89
PCA 3-D latent space	3	0.4079	**0.9934**	11,637.21
PCA 2-D latent space	2	0.4278	0.6920	6,994.91

Dimensional preservation insight. Clustering in the 3-D PCA space yields an ARI of 0.9934 relative to the full 66-D clustering, so three components preserve the cluster structure almost perfectly. Silhouette scores and WCSS values are not directly comparable across spaces of different dimensionality (discarding dimensions removes squared-distance mass by construction, and low-dimensional distances are less contrasted). Compressing further to the 2-D PCA subspace degrades the fidelity: the ARI drops to 0.6920, suggesting that PC3 carries information that is relevant to the cluster assignment.

4.6 Summary of Q3 Findings

Assignment item	Finding
(a) K-Means from scratch, K = 2…6	WCSS = 50,994.77 / 45,671.92 / 41,917.89 / 38,700.20 / 36,900.66 for K = 2…6 (Table 4.5); matches scikit-learn (ARI ≥ 0.9985). Elbow curve in Figure 4.2: soft elbow at K = 4 (K = 3 on the extended grid).
(b) PCA via covariance matrix	Eigenvalues, eigenvectors and explained variance ratios in Tables 4.7–4.8; PC1 28.0%, PC2 14.4%, PC3 11.7%; cumulative curve in Figure 4.4.
(b) Dimensionality reduction	2 PCs retain 42.36% (reconstruction MSE 0.0662); 3 PCs retain 54.10% (MSE 0.0527); 12, 23 and 34 PCs are needed for 80%, 90% and 95%.
(b) Visualisation	Figures 4.5–4.7: PC1 separates movies from TV, PC2 captures recency; clustering in the 3-D PCA space reproduces the full-space partition (ARI 0.9934).

5. Cross-Cutting Discussion

This section collects the conclusions that cut across the three experiments and answers the discussion points of the assignment sheet directly.

5.1 Overfitting, Underfitting and Model Complexity

Each model family exposes a “complexity knob”; Table 5.1 summarises how it was used, what failure appeared at each end of its range, and where the experiments located the optimum.

Table 5.1: Complexity Control and Failure Modes of the Models Studied

Model	Complexity knob	Underfitting end	Overfitting end	Optimum found
KNN	Number of neighbours k	Large k: validation macro-F1 falls toward 0.80	Small k with distance weighting: train score 1.0	k = 4
Decision Tree	Maximum depth / pruning	Depth 1: validation macro-F1 about 0.51	Unpruned (depth 14): 99.94% train vs 90.14% test accuracy	Depth 8
SVM (RBF)	C and γ	C = 0.1, γ = 0.001: CV F1 0.29	C = 1, γ = 1: train–validation gap 0.39	C = 100, γ = 0.03
1D-CNN	Epochs; dropout, weight decay	Early epochs: high loss	After epoch 16: validation loss rises while training loss falls	Early stopping at best validation loss
Polynomial regression	Degree	Degree 1: train R^2 0.19	Degree 8: test RMSE 340,234	Degree 1 (CV RMSE 10.26)
Ridge / Lasso	Penalty α	$\alpha = 10^5$ on polynomials: near-constant predictor	$\alpha \to 0$: OLS-like variance	Ridge α = 2682.7; Lasso α = 1.2068
K-Means	Number of clusters K	K = 2: only movie vs. TV	K large: WCSS falls but gain < 5%	K = 4 (soft elbow; K = 3 extended)
PCA	Number of components k	k = 2: 42.4% variance, ARI 0.69	k = 66: no compression	k = 3 (54.1%, ARI 0.99)

The same bias–variance pattern appears in all three paradigms. In classification, the SVM with the smallest train–test gap (F1 gap 0.1234) also has the best ROC-AUC, whereas the memorising KNN has the largest gap (0.1737). In regression, the more expressive the model (degree 8, 209-term polynomial), the larger the damage on unseen patients, and the strongly shrunk models are the only ones to approach the baseline. In unsupervised learning, "overfitting" is replaced by an over-fine partition with little additional WCSS gain and low silhouette.

5.2 Evaluation Protocol Matters More than Model Choice

- **Class imbalance (Q1).** Overall accuracy ranks the CNN last (0.8826), yet the CNN has the best Pathologic recall (94.3%). Only the per-class tables, macro metrics and OvR ROC analysis expose this trade-off.
- **Patient leakage (Q2).** The same Random Forest scores R^2 = +0.9774 or −1.1656 depending only on how the data were split. Any figure reported without stating the split strategy is uninterpretable on grouped data.
- **Tuning on the evaluation data.** In Q2 the regularisation parameters were tuned on training patients, and the 42-patient CV therefore mildly favours Ridge and Lasso; we state this rather than claim significance.
- **Statistical significance (Q1).** With only 35 Pathologic test cases a 0.013–0.017 AUC difference is within bootstrap noise; the ranking SVM > CNN ≈ KNN is a point estimate, and only the Decision Tree is demonstrably worse.

5.3 The Role of Feature Scaling

Scaling was decisive in all three questions: it enabled gradient descent to converge (Q2: η = 0.001 diverges on raw features but η = 0.1 converges on standardised ones), it is required by the distance-based KNN and kernel-based SVM (Q1, via leakage-safe pipelines), and it governs K-Means and PCA (Q3: only the four continuous features are z-scored so that binary flags are not artificially inflated). The common principle is that an algorithm built on Euclidean geometry inherits the units of its inputs.

5.4 Practical Recommendations

- For tabular classification of this size, prefer a tuned RBF-SVM (or KNN/Decision Tree as baselines) over a deep network; reserve the CNN for data with spatial or temporal structure, or use its class-weighted variant when sensitivity to the rare class dominates.
- For grouped or longitudinal data, always split by group and tune with group-wise cross-validation.
- When multicollinearity is known (VIF ≫ 10), use Ridge for stability or Lasso for selection rather than plain OLS.

- For catalog-like sparse binary data, treat K-Means results as exploratory: report silhouette alongside WCSS, state the soft elbow honestly, and check stability in a PCA subspace.

6. Overall Conclusions and Scientific Limitations

Synthesis of core findings

1. **Classification (Q1).** SVM with an RBF kernel emerges as the best classifier by point estimate (macro ROC-AUC 0.9696, macro-F1 0.8450), although it is not significantly better than KNN or the CNN. Bootstrap hypothesis testing reveals that the Decision Tree is statistically inferior, while KNN and the 1D-CNN achieve parity. The deep 1D-CNN required extensive regularisation to curb memorisation of the tabular data and did not outperform the RBF-SVM.

2. **Regression (Q2).** Scratch batch gradient descent achieves parity with OLS (MSE 93.94 vs 93.90). Severe multicollinearity (VIF 1.0e6) causes explosive OLS variance, which regularisation stabilises. Tuned Ridge/Lasso are marginally better than the mean baseline in patient-wise CV, but the margin is within fold-to-fold noise. Enforcing patient-disjoint splits shows that acoustic features and demographics did not reliably predict total UPDRS for unseen patients, yielding negative test R^2 for all models relative to the baseline mean.

3. **Clustering & PCA (Q3).** From-scratch K-Means and covariance PCA were validated against scikit-learn. Multi-hot catalog representations yield a soft elbow at K = 4 (the extended elbow is K = 3), which cleanly separates TV series and classic cinema from movies; the two large movie clusters (0 and 3) differ mainly in genre mix and overlap substantially. The first 3 principal components account for 54.1% of the catalog variance and give an ARI of 0.993 against the full-dimensional clustering.

Explicit scientific limitations

- **Q1 test-sample limitation.** Although 426 test samples were evaluated, the critical Pathologic class contains only 35 instances. Bootstrap confidence intervals were constructed, but fine-grained obstetric evaluation would require larger multicentre cohorts.
- **Q2 small patient cohort.** The telemonitoring data span 5,875 recordings but originate from only 42 individuals (33 train, 9 test). The wide inter-patient baseline variance makes patient-disjoint extrapolation exceptionally difficult without subject-specific calibration, and a single 9-patient test split is a noisy estimate (hence the additional 42-patient GroupKFold analysis).
- **Q3 absence of ground truth.** Clustering is inherently unsupervised; no objective ground-truth taxonomy exists for Netflix content. Cluster quality and PCA interpretations are contingent on feature-engineering choices (such as the 50-title genre-frequency threshold and the 15-country cut-off).
- **Single-run deep learning.** The CNN was tuned minimally (fixed architecture, five seeds); a broader architecture search could shift its ranking, although the tabular inductive-bias argument suggests it would remain at best on par with the SVM.

7. References

1. UCI Machine Learning Repository: Cardiotocography Dataset (ID 193). https://archive.ics.uci.edu/dataset/193/cardiotocography.
2. Ayres-de-Campos, D., Bernardes, J., Garrido, A., Marques-de-Sá, J., & Pereira-Leite, L. (2000). SisPorto 2.0: A program for automated analysis of cardiotocograms. *Journal of Maternal-Fetal Medicine*, 9(5), 311–318.
3. UCI Machine Learning Repository: Parkinson's Telemonitoring Dataset (ID 189). https://archive.ics.uci.edu/dataset/189/parkinsons+telemonitoring.
4. Tsanas, A., Little, M. A., Ramig, L. O., & Fox, P. (2010). Accurate telemonitoring of Parkinson's disease progression by noninvasive speech tests. *IEEE Transactions on Biomedical Engineering*, 57(4), 884–893.
5. Kaggle Dataset: Shivam Bansal (2021). Netflix Movies and TV Shows Dataset. https://www.kaggle.com/datasets/shivamb/netflix-shows.
6. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
7. Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32*, 8024–8035.
8. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
9. Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.
10. Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. *Proceedings of the 18th ACM-SIAM Symposium on Discrete Algorithms*, 1027–1035.
11. Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*. MIT Press.
12. Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.

13. Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society B*, 58(1), 267–288.
14. Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
15. Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2(1), 193–218.
16. Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations*.
17. Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of ICML*, 448–456.
18. Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A*, 374, 20150202.

Appendix A: Computational Environment and Runtimes

Hardware and software environment.

- Operating system: Windows 11 Enterprise (AMD64 architecture).
- Python runtime: Python 3.14.7 (v3.14.7:d399b2d, Windows).
- Core libraries: PyTorch 2.15.0+cpu, scikit-learn 1.9.1, pandas 3.1.0, numpy 2.5.4, scipy 1.18.1, matplotlib 3.11.2, seaborn 0.13.2, reportlab 5.1.0, ucimlrepo 0.0.7, kagglehub 1.0.3, nbconvert 7.17.2.

Execution times.

Table A.1: Execution Times of the Three Experiment Notebooks

Experiment	Approximate runtime	Dominant cost
Q1 (CTG classification)	~3.8 minutes	Grid searches (KNN 25 s, tree 11 s, SVM 7 s) and CNN training over 5 seeds (137 s)
Q2 (Parkinson's regression)	~19.5 minutes	GroupKFold search over polynomial Ridge/Lasso pipelines
Q3 (Netflix K-Means & PCA)	~1.2 minutes	Multi-start K-Means and eigendecomposition of the 66 × 66 covariance matrix
Total pipeline	**~24.5 minutes**	CPU-only execution (the CNN ran on CPU)

Appendix B: Algorithm Listings

The three from-scratch components are summarised below in pseudocode. Each has been validated numerically (Sections 3.3, 4.2 and 4.4).

B.1 Batch gradient descent for linear regression (Q2a)

```
input : standardised X (n x p), target y, learning rate eta, iterations T
init  : w <- 0, b <- 0
for t = 1 .. T:
    r    <- X w + b - y                  # residuals, shape (n,)
    grad_w <- (1/n) X^T r
    grad_b <- (1/n) sum(r)
    w <- w - eta * grad_w
    b <- b - eta * grad_b
    if loss is NaN or overflows: stop (diverged)
return w, b
```

B.2 Lloyd's K-Means with k-means++ seeding and multi-start (Q3a)

```
for run = 1 .. n_init (=10):
    centroids <- k-means++ seeding (first uniformly, others with prob. proportional to D(x)^2)
    repeat until assignments stop changing:
        D2      <- ||x||^2 - 2 X C^T + ||c||^2          # (n x K) via one matrix product
        labels  <- argmin_k D2
        for k = 1 .. K:
            if cluster k is empty: re-seed centroid at the point with maximal distance
            else: centroid_k <- mean of points with label k
    wcss <- sum_i min_k D2[i, k]
keep the run with the smallest wcss
```

B.3 PCA by covariance eigendecomposition (Q3b)

```
mu    <- mean(X, axis=0);   Xc <- X - mu
Sigma <- Xc^T Xc / (n - 1)                      # (p x p), symmetric PSD
lam, V <- eigh(Sigma)                           # real eigenvalues, orthonormal eigenvectors
order <- argsort(lam, descending);  lam <- lam[order];  V <- V[:, order]
evr   <- lam / sum(lam);   cum <- cumsum(evr)
Z_k   <- Xc V[:, :k]                            # projection onto first k components (k = 2, 3)
X_hat <- Z_k V[:, :k]^T + mu                    # reconstruction; MSE = sum(lam[k:]) / p
```

Appendix C: Supporting Derivations

C.1 Convergence condition of gradient descent

For the quadratic loss J(w) the gradient is $\nabla J = H(w - w^*)$ with $H = (1/n)X^TX$, so the error $e_t = w_t - w^*$ obeys $e_{t+1} = (I - \eta H)e_t$. In the eigenbasis of H each component is multiplied by $(1 - \eta\lambda_i)$ per step, and convergence requires $|1 - \eta\lambda_i| < 1$ for all i, i.e. $0 < \eta < 2/\lambda_{max}$. The speed along the slowest direction is governed by $(1 - \eta\lambda_{min})$, so the number of iterations grows with the condition number $\kappa = \lambda_{max}/\lambda_{min}$. Unscaled features and collinear columns make κ enormous ($\lambda_{min} \approx 0$), explaining both the divergence on raw data (η too large for the huge λ_{max}) and the slow final convergence to the OLS optimum after 50,000 iterations.

C.2 Why Ridge regression stabilises collinear designs

The Ridge solution is $w_{ridge} = (X^TX + \alpha I)^{-1}X^Ty$. Writing the eigendecomposition $X^TX = Q\Lambda Q^T$, the OLS component along eigenvector i is divided by λ_i, which is nearly zero for collinear features and amplifies noise, while Ridge divides by $(\lambda_i + \alpha)$, equivalent to shrinking the component by the factor $\lambda_i/(\lambda_i + \alpha)$. The exploding coefficients of `Shimmer:APQ3` and `Shimmer:DDA` (±780) are therefore replaced by small, shared weights. Lasso's L1 penalty instead selects a sparse subset, here `age` and `PPE`.

C.3 Monotone decrease of the K-Means objective

The assignment step assigns each point to its nearest centroid and therefore cannot increase WCSS. The update step replaces each centroid by the mean of its cluster, which is the minimiser of $\Sigma\|x_i - c\|^2$ over c, and therefore cannot increase WCSS either. Since there are finitely many partitions and the objective is non-increasing, the algorithm terminates at a local minimum; Figure 4.1 (right) shows this monotone behaviour empirically. K-means++ seeding gives an expected cost within a factor O(log K) of the optimum (Arthur and Vassilvitskii, 2007).

C.4 PCA as variance maximisation and the reconstruction error

The first principal axis maximises the projected variance $v^T\Sigma v$ subject to $\|v\| = 1$. Introducing a Lagrange multiplier gives $\Sigma v = \lambda v$, so the optimum is an eigenvector and the achieved variance is its eigenvalue; successive axes repeat the argument in the orthogonal complement. Because the eigenvectors are orthonormal, the squared reconstruction error of the rank-k projection equals the sum of the discarded eigenvalues, so the per-entry MSE is $(\Sigma_{j>k} \lambda_j)/p$, which we verified numerically in §4.4.

C.5 Paired bootstrap for ROC-AUC differences

The test set is resampled with replacement 500 times; on each resample the macro AUC of every model is recomputed on the same resampled cases, and the difference $\Delta AUC = AUC_{model} - AUC_{SVM}$ is recorded. Pairing removes the shared sampling noise, and the 2.5th and 97.5th percentiles of ΔAUC form the 95% interval. An interval containing zero means the data do not demonstrate a difference.

Appendix D: Glossary of Terms and Abbreviations

Term	Meaning
AUC / ROC	Area under the receiver operating characteristic curve; threshold-independent discrimination measure
OvR	One-vs-rest: each class is scored against all others and the results are averaged
Macro average	Unweighted mean of per-class metrics, giving minority classes equal weight
CTG / NSP	Cardiotocography / Normal–Suspect–Pathologic fetal state label
UPDRS	Unified Parkinson's Disease Rating Scale (clinician-rated severity)
VIF	Variance inflation factor, $1/(1 - R^2_j)$; values above 10 indicate problematic collinearity
GroupKFold	Cross-validation in which all rows of one group (patient) fall into the same fold
Ridge / Lasso	L2 / L1 penalised least squares
1-SE rule	Choose the most regularised model whose CV error is within one standard error of the minimum
WCSS	Within-cluster sum of squares (K-Means inertia)
Silhouette	Cluster cohesion/separation score in [−1, 1]
ARI	Adjusted Rand Index; chance-corrected agreement between two partitions
EVR	Explained variance ratio of a principal component
PCA	Principal component analysis; orthogonal projection onto maximal-variance directions

Appendix E: Submitted Code and Output Files

The code is submitted as a zip archive alongside this PDF. The archive is organised as follows; the code and results are those of the original runs, and no additional runs were made for this report. The three notebooks reproduce every table and figure.

```
ML-Assignment/
  data/        README.txt
  figures/     q1/ (12 figures)   q2/ (13 figures)   q3/ (9 figures)
  notebooks/   q1_classification.ipynb   q2_regression.ipynb   q3_kmeans_pca.ipynb
  results/     q1_*.csv/.json   q2_*.csv/.json   q3_*.csv/.json/.npy
  .gitignore   CHANGES.md   CHECKS.md   README.md   report.pdf   requirements.txt
```

Path	Content
`data/README.txt`	Instructions for obtaining the three datasets (UCI CTG, UCI Parkinson's Telemonitoring, Kaggle Netflix titles)
`notebooks/q1_classification.ipynb`	Q1: CTG preprocessing, KNN/Decision Tree/SVM grid searches, 1D-CNN (5 seeds), ROC and bootstrap analysis, overfitting study

Path	Content
notebooks/q2_regression.ipynb	Q2: patient-wise split, leakage experiment, VIF audit, scratch gradient descent, polynomial study, Ridge/Lasso CV, benchmark synthesis
notebooks/q3_kmeans_pca.ipynb	Q3: Netflix feature engineering, scratch K-Means and PCA, validation vs. scikit-learn, elbow/silhouette, cluster profiling, PCA-space clustering
results/q1_*.csv, .json	Q1 test/train metrics, best parameters, AUC confidence intervals, CNN seed stability (Table 2.6)
results/q2_*.csv, .json	Q2 coefficients (Section 3.2), VIF, GroupKFold robustness (Table 3.8), leakage test (Table 3.1)
results/q3_*.csv, .json, .npy	Q3 WCSS/elbow (Table 4.5), PCA loadings and variance (Tables 4.7–4.9), cluster profiles (Table 4.6), 2-D/3-D embeddings
figures/q1, figures/q2, figures/q3	12, 13 and 9 figures used in Sections 2, 3 and 4 respectively
README.md, CHECKS.md, CHANGES.md	Run instructions, verification checks and change log of the repository
requirements.txt	Python package requirements (see Appendix A for the versions used)
report.pdf	This report